

THE COMPLETE

SQL MASTERY

INTERVIEW BIBLE

HackerRank · LeetCode · FAANG · Service-Based · Big Tech · All Companies
Beginner to Expert · Visual Diagrams · Wrong→Right Examples · Company Strategy

200+

- ◆ Step-by-step from ZERO to SQL Expert — Beginner-friendly language throughout
- ◆ Visual diagrams: JOINS · Execution Order · NULL · Window vs GROUP · Gap & Island · Running Total
 - ◆ Every HackerRank SQL question + full LeetCode SQL 50
- ◆ ■ Wrong Approach shown before ■ Right Approach — learn from mistakes
- ◆ FAANG strategy (Google · Amazon · Meta · Microsoft · Uber · Netflix · Stripe)
 - ◆ Service-based strategy (TCS · Infosys · Wipro · Accenture · Cognizant)
- ◆ Top 50 Most Frequently Asked SQL Questions across ALL companies — with answers
 - ◆ Interview scripts: exactly what to SAY when writing SQL

DEFINITIVE EDITION · 2025–2026

READ FIRST

How to Use This Book

Your exact roadmap — follow this and you WILL crack SQL interviews

■ *Real story: A student who had never written SQL in his life used this book for 2 weeks. He got selected at Infosys, then 3 months later cracked Amazon. His strategy: 'I read each question, looked at the wrong approach, understood WHY it fails, then wrote the correct solution 3 times without looking.' That is the method. Follow it exactly.*

WHEN	WHAT TO DO	HOW TO DO IT
DAY 1–2	Read Section 1 (Theory) completely	90% of telephonic rounds at ALL companies ask theory. Read each concept aloud in your own words. If you can't explain it simply, you don't understand it yet.
DAY 3–7	Study HackerRank (Sections 2–6)	For each question: (1) Read the problem. (2) Look at the ■ Wrong approach — understand WHY it fails. (3) Study the ■ correct solution. (4) Close the book and write it yourself.
DAY 8–12	Study LeetCode SQL 50 (Section 7)	These cover every pattern used in real interviews. Don't just read — identify which of the 15 patterns each question uses before looking at the solution.
DAY 13–14	Study Company-Specific Sections (8,9)	Section 8 = FAANG strategy and questions. Section 9 = Service company questions. Read the company strategies carefully — they explain what each company actually tests.
DAY 15	Memorize the Master Cheat Sheet (Section 10)	The 15 patterns, NULL rules, execution order, window functions reference. This one page will save you in every interview.
DAY 16+	Mock interview yourself DAILY	Open a blank editor. Pick a random question from this book. Set a 15-minute timer. Write SQL and explain your thinking out loud. This is the only way to build real confidence.

■■ WATCH OUT

THE GOLDEN RULE: Reading SQL ≠ Knowing SQL. You MUST write every solution yourself, from scratch, without looking. Candidates who can only read solutions FAIL interviews. Candidates who can write from memory PASS. There is no shortcut to this.

■ INTERVIEW TRICK

INTERVIEW SCRIPT: When shown a SQL problem say out loud: 'Let me identify the pattern first. This looks like a [Top-N / Running Total / Gap & Island] problem. I'll use [DENSE_RANK / SUM OVER / date subtraction] because... One edge case to consider: what if there are NULLs / ties / empty table?' This structured thinking impresses EVERY interviewer at every company.

Table of Contents

SEC	SECTION	WHAT YOU MASTER	Qs
1	SQL Fundamentals & Theory — with visual diagrams	Concepts, ACID, Normalization, JOINS, NULLs, Indexes, Views — 15 theory questions asked in every telephonic round	15
2	HackerRank Basic Select	SELECT, WHERE, DISTINCT, REGEXP, ORDER BY — 20 questions with wrong→right	20
3	HackerRank Advanced Select	CASE, Pivot (visual!), Binary Tree, The PADS	5
4	HackerRank Aggregation	COUNT/SUM/AVG, Blunder, Median, Distance formulas	17
5	HackerRank Basic Join	INNER/LEFT/RANGE JOINS — visual reference + 8 questions	8
6	HackerRank Advanced Join & Alternatives	Gap & Island (visual!), Placements, Interviews, 15 Days, Prime Numbers	8
7	LeetCode SQL 50	All 50 official problems with Top-N, Running Total, Anti-Join diagrams	50
8	FAANG & Big Tech Strategy	Company-specific strategy + real questions: Google, Amazon, Meta, Microsoft, Uber, Netflix, Stripe, Flipkart	25+
9	Service-Based Company Strategy	Company-specific strategy + real questions: TCS, Infosys, Wipro, Accenture, Cognizant, Capgemini	15+
10	Top 50 Most Frequently Asked SQL Questions	The 50 questions asked most often across ALL companies worldwide — with complete answers	50
11	THE MASTER CHEAT SHEET	15 patterns · Visual diagrams · Window functions · NULL rules · Index rules · Interview scripts · Checklist	—

SECTION 1

SQL Fundamentals

& Theory

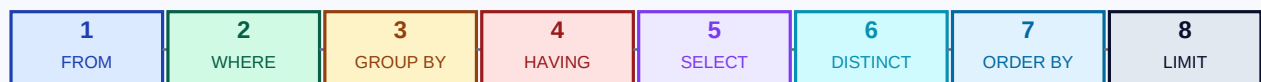
Visual diagrams + 15 theory questions — master this before writing any code

■ BEGINNER NOTE

If you are completely new to SQL: SQL = Structured Query Language. It is the language used to talk to databases. Every company in the world stores data in databases. SQL is how you get that data out, transform it, and analyze it. Learning SQL = learning to speak to data.

Visual 1: SQL Execution Order

SQL Execution Order — SQL Does NOT Execute Top to Bottom!



■ You WRITE: SELECT...FROM...WHERE — SQL READS: FROM first, WHERE second, SELECT fifth!

■ MEMORY TRICK

Mnemonic: 'Friendly Whales Generally Have Sweet Dreams Often Late' → FROM, WHERE, GROUP BY, HAVING, SELECT, DISTINCT, ORDER BY, LIMIT

■ Using SELECT alias in WHERE (impossible!)

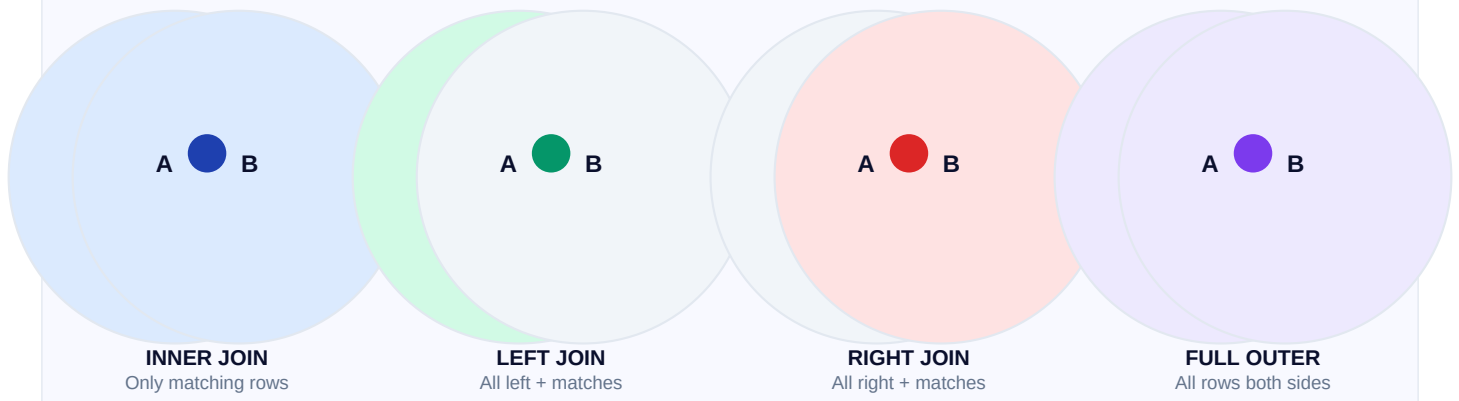
```
SELECT salary * 12 AS annual,
WHERE annual > 500000 -- ■ ERROR!
-- annual doesn't exist yet when WHERE runs
```

■ Use the actual expression in WHERE

```
SELECT salary * 12 AS annual,
FROM Employee
WHERE salary * 12 > 500000 -- ■
-- or put it in HAVING after GROUP BY
```

Visual 2: SQL JOIN Types

SQL JOIN Types — Visual Guide



■ **INNER JOIN** loses rows with no match

```
-- Loses employees with no department!
SELECT e.name, d.dept_name
FROM   Employee e
INNER JOIN Department d ON e.dept_id=d.id
```

■ **LEFT JOIN** keeps all rows from left table

```
-- Keeps ALL employees, NULL if no dept
SELECT e.name, d.dept_name
FROM   Employee e
LEFT JOIN Department d ON e.dept_id=d.id
```

Visual 3: NULL — The #1 Source of SQL Bugs

NULL — SQL Uses 3-Valued Logic: TRUE / FALSE / NULL

NULL = NULL	→ NULL (not TRUE!) Use IS NULL
NOT IN with NULL	→ EMPTY RESULT! Use NOT EXISTS
FALSE AND NULL	→ FALSE
COUNT(*)	→ counts NULLs. COUNT(col) skips NULLs

5 + NULL	→ NULL. Any math with NULL = NULL
TRUE AND NULL	→ NULL (unknown)
TRUE OR NULL	→ TRUE
COALESCE(a, b)	→ returns first non-NULL value

■ **NOT IN** trap with NULLs

```
-- WRONG: returns 0 rows if any NULL!
SELECT * FROM Employee
WHERE dept_id NOT IN (
  SELECT id FROM Department
  -- If id has ANY NULL → empty result!
);
```

■ **NOT EXISTS** (always NULL-safe)

```
-- CORRECT: NOT EXISTS is NULL-safe
SELECT * FROM Employee e
WHERE NOT EXISTS (
  SELECT 1 FROM Department d
  WHERE d.id = e.dept_id
);
```

Visual 4: Window Functions vs GROUP BY

GROUP BY Collapses Rows vs Window Function Keeps All Rows

■ GROUP BY (rows collapse)

Alice	Sales	70,000
Bob	Sales	80,000
Carol	Eng	90,000

▼ GROUP BY dept

Sales	avg 75,000
Eng	avg 90,000

Individual names are GONE!

■ WINDOW FUNCTION (rows kept)

Alice	70,000	avg=75,000
Bob	80,000	avg=75,000
Carol	90,000	avg=90,000

All rows + new dept_avg column!

■ KEY PATTERN

RULE: If you need individual rows AND aggregate values at the same time → Window Function. If you only need summary data → GROUP BY.

Visual 5: Gap & Island Pattern

Gap & Island — How Subtracting ROW_NUMBER Creates Group Keys

Login Date	ROW_NUMBER()	Date minus RN(days)	Group = Island
Jan 1	1	Dec 31	Island 1
Jan 2	2	Dec 31	Island 1
Jan 3	3	Dec 31	Island 1
Jan 5	4	Jan 1	Island 2
Jan 6	5	Jan 1	Island 2
Jan 9	6	Jan 3	Island 3

Same group key = consecutive dates = one island/streak. Gap in dates = different key = new island.

■ MEMORY TRICK

Gap & Island: subtract ROW_NUMBER from the date/value. Consecutive same values cancel out → same group key. A gap → different group key. One pattern, many uses: login streaks, absent days, price periods, project tasks.

Visual 6: Top-N Per Group

Top-N Per Group — The Most Frequently Asked Window Function Pattern

Name	Dept	Salary	DENSE_RANK() per Dept	Keep? (rn<=3)
Alice	Sales	90,000	rn=1	YES
Bob	Sales	80,000	rn=2	YES
Carol	Sales	70,000	rn=3	YES
Dave	Sales	60,000	rn=4	NO
Eve	Eng	95,000	rn=1	YES
Frank	Eng	85,000	rn=2	YES
Grace	Eng	75,000	rn=3	YES
Hank	Eng	65,000	rn=4	NO

■ KEY PATTERN

Top-N per group = ROW_NUMBER() OVER (PARTITION BY group ORDER BY value DESC), then WHERE rn <= N. Use DENSE_RANK when you want to include tied values at position N.

Visual 7: Running Total

Running Total — SUM() OVER (ORDER BY date ROWS UNBOUNDED PRECEDING)

Date	Revenue	Running Total
Jan 1	1,000	1,000
Jan 2	2,500	3,500
Jan 3	500	4,000
Jan 4	3,000	7,000
Jan 5	1,500	8,500

■ PRO TIP

SUM(col) OVER (ORDER BY date ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW) = cumulative sum. Always specify ROWS (not RANGE) for running totals to avoid issues with same-date rows.

15 Theory Questions — Asked in Every Telephonic Round

Q: DELETE vs TRUNCATE vs DROP

A: DELETE: DML, removes rows one-by-one, logged, supports WHERE clause, ROLLBACK possible. TRUNCATE: DDL, removes ALL rows instantly, minimal logging, resets AUTO_INCREMENT, no WHERE, NO rollback in MySQL (yes in PostgreSQL). DROP: removes the entire table — structure, data, indexes — permanently.

■ MEMORY TRICK

Memory: D=row(Delete), T=Table rows(Truncate), D=Done/Table(Drop). Each is more destructive.

■■ WATCH OUT

MySQL: TRUNCATE cannot be rolled back. PostgreSQL: TRUNCATE CAN (it is transactional). Know your database!

Q: What are ACID properties?

A: Atomicity: all operations in a transaction succeed or all roll back (all-or-nothing). Consistency: DB always moves from one valid state to another (constraints always satisfied). Isolation: concurrent transactions cannot see each other's intermediate state. Durability: once committed, data survives crashes (written to disk).

■ MEMORY TRICK

Real example: Bank transfer ■1000 from A to B. Both debit AND credit must succeed. If credit fails, debit rolls back. That is Atomicity.

■ INTERVIEW TRICK

Isolation levels weakest→strongest: READ UNCOMMITTED → READ COMMITTED → REPEATABLE READ → SERIALIZABLE. Higher = fewer bugs but more locking overhead.

Q: What is Normalization? Explain 1NF, 2NF, 3NF.

A: 1NF: Atomic values (no lists in a cell), each column one data type, unique rows. 2NF: 1NF + no partial dependency (non-key columns must depend on the WHOLE primary key). 3NF: 2NF + no transitive dependency (non-key col A → non-key col B is not allowed). BCNF: every determinant must be a candidate key.

■ MEMORY TRICK

The golden rule of 3NF: Every non-key attribute must depend on THE KEY, THE WHOLE KEY, AND NOTHING BUT THE KEY.

■ INTERVIEW TRICK

Denormalization = intentionally breaking normalization for query performance. Data warehouses (analytics) are often denormalized for faster reads.

Q: PRIMARY KEY vs UNIQUE KEY vs FOREIGN KEY

A: PRIMARY KEY: uniquely identifies each row, NOT NULL enforced, only ONE per table, creates clustered index.

UNIQUE KEY: ensures uniqueness, allows ONE NULL per column, multiple per table, non-clustered index. FOREIGN

KEY: references PK of another table, enforces referential integrity (CASCADE, SET NULL, RESTRICT).

■ MEMORY TRICK

PK = Aadhaar card (unique, mandatory). UK = PAN card (unique, optional). FK = your employee ID referencing the HR table.

Q: WHERE vs HAVING

A: WHERE filters individual ROWS before any grouping — runs on raw data, aggregate functions NOT allowed.

HAVING filters GROUPS after GROUP BY — runs on grouped data, aggregate functions required for meaningful use.

■ PRO TIP

Simple rule: condition uses COUNT/SUM/AVG/MIN/MAX? → HAVING. Otherwise → WHERE.

■■ WATCH OUT

Common mistake: WHERE COUNT(*) > 5 → ERROR. Fix: GROUP BY dept HAVING COUNT(*) > 5.

Q: What is an Index? Types?

A: An index is a data structure (B-Tree by default) that allows the database to find rows faster without scanning the whole table. Types: B-Tree (default, works for =, <, >, BETWEEN, LIKE 'prefix%'), Hash (equality only), Composite (multiple columns), Clustered (data physically sorted by this key), Covering (all needed columns in index).

■ MEMORY TRICK

Index = book index. Without index, DB reads every page (full scan). With index, DB jumps directly to the right page.

■■ WATCH OUT

Composite index LEFT-PREFIX RULE: index on (A,B,C) helps queries on A, A+B, A+B+C. Query on B alone does NOT use this index.

Q: Views vs Materialized Views

A: VIEW: virtual table — no data stored, query runs fresh every time. Always current data, can be slow. MATERIALIZED

VIEW: physical snapshot stored on disk, very fast reads, must be refreshed manually/scheduled, may be slightly stale.

■ MEMORY TRICK

View = TV stream (always live). Materialized View = recorded video (fast to watch but might be old).

■ INTERVIEW TRICK

MySQL has no native MATERIALIZED VIEW. Simulate with: CREATE TABLE summary AS SELECT... + scheduled job to refresh.

Q: UNION vs UNION ALL

A: UNION: combines two SELECT results, removes duplicates (requires internal sort/hash). Slower. UNION ALL: combines results, keeps ALL rows including duplicates. No dedup step. Faster.

■ PRO TIP

ALWAYS prefer UNION ALL unless you explicitly NEED deduplication. UNION ALL can be 2–10x faster.

■■ WATCH OUT

Both queries must have same number of columns with compatible data types. Column names come from the FIRST SELECT.

Q: What is a CTE (WITH clause)?

A: A CTE (Common Table Expression) is a named temporary result set defined at the start of a query. Benefits: improves readability, replaces complex subqueries, can be referenced multiple times. Recursive CTEs traverse hierarchical data (org charts, trees).

■ PRO TIP

CTE vs Subquery: CTEs are named and reusable within the query. Both have similar performance in most engines. CTEs are easier to debug.

■ INTERVIEW TRICK

Recursive CTE structure: ANCHOR (base case) UNION ALL RECURSIVE (self-reference with termination condition).

Q: What are Window Functions?

A: Window functions perform calculations across a related set of rows WITHOUT collapsing them into one row. Syntax: function() OVER (PARTITION BY col ORDER BY col ROWS BETWEEN frame). PARTITION BY = GROUP BY equivalent but rows are NOT removed. ORDER BY inside OVER = ordering within partition. ROWS BETWEEN = window frame.

■ MEMORY TRICK

ROW_NUMBER = unique (1,2,3). RANK = gaps after ties (1,1,3). DENSE_RANK = no gaps (1,1,2). Use DENSE_RANK for top-N salary problems.

■■ WATCH OUT

Window functions execute AFTER WHERE and GROUP BY but BEFORE ORDER BY. Cannot filter window results in WHERE — wrap in CTE or subquery.

Q: What is a Stored Procedure vs Function?

A: Stored Procedure: CALL statement, can return multiple result sets via OUT params, CAN do DML (INSERT/UPDATE/DELETE), cannot be used in SELECT. Function: returns exactly ONE value, used INSIDE SELECT/WHERE, should be deterministic (same input = same output).

■ MEMORY TRICK

Function = math function $f(x)=y$ (pure, returns value). Procedure = recipe/workflow (does things step by step).

Q: Explain INNER JOIN, LEFT JOIN, RIGHT JOIN, FULL OUTER JOIN.

A: INNER JOIN: only rows that match in BOTH tables. LEFT JOIN: ALL rows from left + matching from right (NULL if no match). RIGHT JOIN: ALL rows from right + matching from left (NULL if no match). FULL OUTER JOIN: ALL rows from BOTH sides (NULL where no match).

■ KEY PATTERN

Anti-join pattern: LEFT JOIN + WHERE right_table.id IS NULL = rows in left table with NO match in right. Faster and safer than NOT IN.

Q: What is a Self Join?

A: A self join is when a table is joined to itself. Useful for: comparing rows within the same table, hierarchies (employee-manager), finding related records. You MUST use aliases to distinguish the two 'copies' of the table.

■ PRO TIP

Example: Find employees earning more than their manager. Join Employee e1 (employee) with Employee e2 (manager) on e1.managerId = e2.id.

Q: What is the difference between CHAR and VARCHAR?

A: CHAR(n): fixed-length, always stores exactly n bytes. Faster for fixed-size data (country codes, status flags). VARCHAR(n): variable-length, stores only what's needed + 1-2 byte overhead. Flexible for variable-length data. TEXT: for large text, not fully indexable.

■ PRO TIP

Use CHAR for: country codes ('IN','US'), status ('Y','N'), fixed-length hashes. Use VARCHAR for: names, emails, addresses. Never use TEXT for JOIN/WHERE columns.

Q: What is a Correlated vs Non-Correlated Subquery?

A: Non-correlated subquery: runs ONCE independently, result passed to outer query. Fast. Correlated subquery: references the outer query's columns, runs ONCE PER ROW of outer query. Can be very slow on large tables. Replace correlated subqueries with JOINS or CTEs whenever possible.

■ INTERVIEW TRICK

Correlated subquery test: can the inner query run independently without the outer query? If NO → correlated. Replace with JOIN for better performance.

■ PRO TIP

EXISTS is usually implemented as a correlated subquery but short-circuits on first match — often faster than IN for large subquery results.

— ■ PRACTICE: Without looking, explain DELETE vs TRUNCATE vs DROP in under 60 seconds. | Explain what ACID means with a real-world example.

HackerRank

All 58 Questions

Basic Select · Advanced Select · Aggregation · Basic Join · Advanced Join

■ KEY PATTERN

All 58 HackerRank SQL questions. Tables used: CITY(ID,Name,CountryCode,District,Population) and STATION(ID,CITY,STATE,LAT_N,LONG_W). Memorize these schemas — they appear in 37 of the 58 questions.

#1 Revising Select Query I

Basic SELECT

● EASY

■ **Problem:** Query ALL columns from CITY: CountryCode='USA' AND Population > 100000.

■ **Tables:** CITY(ID,Name,CountryCode,District,Population)

■ **Solution:**

```
SELECT * FROM CITY
WHERE CountryCode='USA' AND Population>100000;
```

■ PRO TIP

Composite index on (CountryCode, Population): equality first, range second — optimal.

#2 Revising Select Query II

Basic SELECT

● EASY

■ **Problem:** Query only CITY Name: CountryCode='USA' AND Population > 120000.

■ **Solution:**

```
SELECT Name FROM CITY
WHERE CountryCode='USA' AND Population>120000;
```

■ PRO TIP

Always select only needed columns in production code, not SELECT *.

#3 Select All

Basic SELECT

● EASY

■ **Problem:** Query all columns for every row in CITY.

■ **Solution:**

```
SELECT * FROM CITY;
```

■ BEGINNER NOTE

SELECT * is fine in interviews. In production: always name columns explicitly.

#4 Select By ID

Basic SELECT

● EASY

■ **Problem:** Query all columns for city with ID = 1661.

■ **Solution:**

```
SELECT * FROM CITY WHERE ID=1661;
```

■ PRO TIP

Primary key lookup = $O(1)$ with B-Tree index. Fastest possible query.

#5 Japanese Cities Attributes

Basic SELECT

● EASY

■ **Problem:** All attributes of Japanese cities (CountryCode='JPN').

■ **Solution:**

```
SELECT * FROM CITY WHERE CountryCode='JPN';
```

#6 Japanese Cities Names

Basic SELECT

● EASY

■ **Problem:** Names of all Japanese cities, sorted alphabetically.

■ **Solution:**

```
SELECT Name FROM CITY WHERE CountryCode='JPN' ORDER BY Name ASC;
```

■ INTERVIEW TRICK

Always explicitly write ASC/DESC — signals precision to interviewers.

#7 Weather Observation Station 1

Basic SELECT

● EASY

■ **Problem:** Query CITY and STATE from STATION.

■ **Tables:** STATION(ID,CITY,STATE,LAT_N,LONG_W)

■ **Solution:**

```
SELECT CITY, STATE FROM STATION;
```

■ MEMORY TRICK

STATION also has LAT_N and LONG_W — used in aggregation questions. Memorize the schema!

#8 Weather Observation Station 3

DISTINCT + MOD

● EASY

■ **Problem:** Distinct CITY names with even-numbered IDs.

■ **Solution:**

```
SELECT DISTINCT CITY FROM STATION
WHERE MOD(ID,2)=0 ORDER BY CITY;
```

■ MEMORY TRICK

MOD(ID,2)=0 and ID%2=0 are identical. DISTINCT adds a sort/dedup step.

#9 Weather Observation Station 4

COUNT vs COUNT DISTINCT

● EASY

■ **Problem:** Difference between total CITY count and count of distinct CITY names.

■ **Solution:**

```
SELECT COUNT(CITY)-COUNT(DISTINCT CITY) FROM STATION;
```

■ KEY PATTERN

This equals the number of duplicate city name entries — a clever deduplication check.

#10 Weather Observation Station 5

UNION ALL + LENGTH

● EASY

-- WRONG: no parentheses =
ORDER BY ignored
per-SELECT,
SELECT
CITY,LENGTH(CITY)
FROM STATION
ORDER BY
LENGTH(CITY) ASC
LIMIT 1, UNION ALL,
SELECT
CITY,LENGTH(CITY)
FROM STATION
ORDER BY
LENGTH(CITY)
DESC LIMIT 1;

■ **Problem:** City with shortest name AND city with longest name. Alphabetical tie-break.

■ **Solution:**

```
(SELECT CITY,LENGTH(CITY) len FROM STATION ORDER BY len ASC,CITY ASC LIMIT 1)
UNION ALL
(SELECT CITY,LENGTH(CITY) len FROM STATION ORDER BY len DESC,CITY ASC LIMIT 1);
```

■ WATCH OUT

Parentheses allow ORDER BY + LIMIT inside each SELECT before UNION ALL. Without parentheses, ORDER BY applies to combined result!

#11 Weather Observation Station 6

REGEXP

● EASY

■ **Problem:** Distinct city names starting with a vowel.

■ **Solution:**

```
SELECT DISTINCT CITY FROM STATION WHERE CITY REGEXP '^[AEIOUaeiou]';
```

■ MEMORY TRICK

^ = start of string. [AEIOUaeiou] = any vowel. \$ = end of string. .* = any characters.

#12 Weather Observation Station 7

REGEXP

● EASY

■ **Problem:** Distinct city names ending with a vowel.

■ **Solution:**

```
SELECT DISTINCT CITY FROM STATION WHERE CITY REGEXP '[AEIOUaeiou]$';
```

#13 Weather Observation Station 8

REGEXP

● EASY

■ **Problem:** Cities starting AND ending with vowel.

■ **Solution:**

```
SELECT DISTINCT CITY FROM STATION WHERE CITY REGEXP '^[AEIOUaeiou].*[AEIOUaeiou]$';
```

■ PRO TIP

.* matches any characters in between. Works for single-vowel and multi-character cities.

#14 Weather Observation Station 9

NOT REGEXP

● EASY

■ **Problem:** Cities NOT starting with vowel.

■ **Solution:**

```
SELECT DISTINCT CITY FROM STATION WHERE CITY NOT REGEXP '^[AEIOUaeiou]';
```

#15 Weather Observation Station 10

NOT REGEXP

● EASY

■ **Problem:** Cities NOT ending with vowel.

■ **Solution:**

```
SELECT DISTINCT CITY FROM STATION WHERE CITY NOT REGEXP '[AEIOUaeiou]$';
```

#16 Weather Observation Station 11

De Morgan OR

● EASY

■ **Problem:** Cities NOT starting OR NOT ending with vowel.

■ **Solution:**

```
SELECT DISTINCT CITY FROM STATION
WHERE CITY NOT REGEXP '^([AEIOUaeiou]
      OR CITY NOT REGEXP '[AEIOUaeiou]$';
```

■ KEY PATTERN

De Morgan's Law: $\text{NOT}(A \text{ AND } B) = \text{NOT}(A) \text{ OR } \text{NOT}(B)$. Stations 6–12 form a logical set — learn them together.

#17 Weather Observation Station 12

AND NOT

● EASY

■ **Problem:** Cities NOT starting AND NOT ending with vowel.

■ **Solution:**

```
SELECT DISTINCT CITY FROM STATION
WHERE CITY NOT REGEXP '^([AEIOUaeiou]
      AND CITY NOT REGEXP '[AEIOUaeiou]$';
```

#18 Higher Than 75 Marks

RIGHT() + ORDER BY

● EASY

■ **Problem:** Student names with Marks > 75. Order by last 3 chars of Name, then ID ASC.

■ **Tables:** Students(ID,Name,Marks)

■ **Solution:**

```
SELECT Name FROM Students
WHERE Marks>75
ORDER BY RIGHT(Name,3) ASC, ID ASC;
```

■ PRO TIP

RIGHT(Name,3) extracts last 3 characters. For large tables: add generated column for index support.

#19 Employee Names

ORDER BY

● EASY

■ **Problem:** All employee names sorted alphabetically.

■ **Tables:** Employee(name,salary,months)

■ **Solution:**

```
SELECT name FROM Employee ORDER BY name ASC;
```

#20 Employee Salaries

WHERE AND

● EASY

■ **Problem:** Names of employees: salary > 2000 AND months < 10. Sort by name.

■ **Solution:**

```
SELECT name FROM Employee
WHERE salary>2000 AND months<10
ORDER BY name ASC;
```

■ **PRO TIP**

Covering index (salary,months,name): all 3 columns in one index. No heap lookup needed.

#21 Type of Triangle

CASE

● EASY

-- WRONG: checks
Equilateral before
validity, CASE
WHEN A=B AND
B=C THEN
'Equilateral' --
(1,1,10) passes!,
WHEN A+B<=C ...
THEN 'Not A
Triangle'

■ **Problem:** Classify each triangle row: Equilateral/Isosceles/Scalene/Not A Triangle.

■ **Tables:** TRIANGLES(A,B,C)

■ **Solution:**

```
SELECT CASE
  WHEN A+B<=C OR A+C<=B OR B+C<=A THEN 'Not A Triangle'
  WHEN A=B AND B=C THEN 'Equilateral'
  WHEN A=B OR B=C OR A=C THEN 'Isosceles'
  ELSE 'Scalene' END AS type
FROM TRIANGLES;
```

■■ **WATCH OUT**

ALWAYS check Not A Triangle FIRST. CASE evaluates top-to-bottom — if (1,1,10) hits Equilateral check first, it would wrongly qualify!

#22 The PADS

CONCAT + GROUP BY

● EASY

■ **Problem:** Name(first_letter) sorted by name; count sentences per occupation.

■ **Tables:** OCCUPATIONS(Name,Occupation)

■ **Solution:**

```
SELECT CONCAT(Name, '(', SUBSTRING(Occupation, 1, 1), ')') FROM OCCUPATIONS ORDER BY Name;
SELECT CONCAT('There are a total of ', COUNT(*), ' ', LOWER(Occupation), 's.')
FROM OCCUPATIONS GROUP BY Occupation ORDER BY COUNT(*) ASC, Occupation ASC;
```

■ **PRO TIP**

SUBSTRING(str,1,1)=first char. LOWER()=lowercase. The 's.' pluralizes. ORDER BY COUNT then Occupation for tie-break.

#23 Occupations (Pivot)

ROW_NUMBER PIVOT

◆ MEDIUM

■ **Problem:** Pivot OCCUPATIONS: each Name alphabetically under Doctor/Professor/Singer/Actor.

■ **Tables:** OCCUPATIONS(Name,Occupation)

■ **Solution:**

```
WITH Ranked AS (  
    SELECT Name, Occupation,  
           ROW_NUMBER() OVER (PARTITION BY Occupation ORDER BY Name) AS rn  
    FROM    OCCUPATIONS  
)  
SELECT MAX(CASE WHEN Occupation='Doctor'    THEN Name END) AS Doctor,  
       MAX(CASE WHEN Occupation='Professor' THEN Name END) AS Professor,  
       MAX(CASE WHEN Occupation='Singer'    THEN Name END) AS Singer,  
       MAX(CASE WHEN Occupation='Actor'     THEN Name END) AS Actor  
FROM    Ranked GROUP BY rn ORDER BY rn;
```

■ KEY PATTERN

ROW_NUMBER per occupation creates row index 1,2,3... Rows with same rn go in same output row. MAX collapses NULLs to keep the Name.

■ MEMORY TRICK

This CASE+MAX pivot works in ALL databases. No PIVOT keyword needed.

#24 Binary Tree Nodes

CASE Subquery

◆ MEDIUM

-- WRONG: NOT IN
without NULL filter,
WHEN N NOT IN
(SELECT P FROM
BST) -- P contains
NULL!, -- Returns
empty because NOT
IN + NULL = NULL =
excluded

■ **Problem:** Label each BST node: Root/Leaf/Inner.

■ **Tables:** BST(N INT, P INT)

■ **Solution:**

```
SELECT N, CASE  
    WHEN P IS NULL THEN 'Root'  
    WHEN N NOT IN (SELECT P FROM BST WHERE P IS NOT NULL) THEN 'Leaf'  
    ELSE 'Inner' END AS NodeType  
FROM BST ORDER BY N;
```

■ WATCH OUT

WHERE P IS NOT NULL in subquery is CRITICAL. NOT IN with NULL = empty result! Root node has P=NULL.

#25 New Companies

Multi-JOIN COUNT DISTINCT

◆ MEDIUM

■ **Problem:** company_code, founder, distinct counts of managers/employees.

■ **Tables:** Company, Lead_Manager, Senior_Manager, Manager, Employee tables

■ **Solution:**

```
SELECT c.company_code, c.founder,
       COUNT(DISTINCT lm.lead_manager_code),
       COUNT(DISTINCT sm.senior_manager_code),
       COUNT(DISTINCT m.manager_code),
       COUNT(DISTINCT e.employee_code)
FROM   Company c
JOIN   Lead_Manager lm ON c.company_code=lm.company_code
JOIN   Senior_Manager sm ON c.company_code=sm.company_code
JOIN   Manager m ON c.company_code=m.company_code
JOIN   Employee e ON c.company_code=e.company_code
GROUP BY c.company_code, c.founder ORDER BY c.company_code;
```

■ ■ **WATCH OUT**

COUNT(DISTINCT) handles duplicates the problem mentions. company_code is STRING — 'C_1' < 'C_10' < 'C_2' lexicographically!

#26 Rev. Aggregations – Count

COUNT

● EASY

■ **Problem:** Count cities with Population > 100000.

■ **Solution:**

```
SELECT COUNT(*) FROM CITY WHERE Population>100000;
```

■ **BEGINNER NOTE**

COUNT(*) counts NULLs. COUNT(col) skips NULLs.

#27 Rev. Aggregations – Sum

SUM

● EASY

■ **Problem:** Total population where District='California'.

■ **Solution:**

```
SELECT SUM(Population) FROM CITY WHERE District='California';
```

#28 Rev. Aggregations – Averages

AVG

● EASY

■ **Problem:** Average population where District='California'.

■ **Solution:**

```
SELECT AVG(Population) FROM CITY WHERE District='California';
```

#29 Average Population

FLOOR + AVG

● EASY

■ **Problem:** Average population for ALL cities, rounded DOWN.

■ **Solution:**

```
SELECT FLOOR(AVG(Population)) FROM CITY;
```

■ **MEMORY TRICK**

FLOOR=round down always. CEIL=round up always. ROUND=nearest. TRUNCATE=chop.

#30 Japan Population

SUM

● EASY

■ **Problem:** Sum of population for Japan (CountryCode='JPN').

■ **Solution:**

```
SELECT SUM(Population) FROM CITY WHERE CountryCode='JPN';
```

#31 Population Density Difference

MAX-MIN

● EASY

■ **Problem:** Difference between max and min population.

■ **Solution:**

```
SELECT MAX(Population)-MIN(Population) FROM CITY;
```

#32 The Blunder

CEIL REPLACE AVG

● EASY

■ **Problem:** CEIL of (correct_avg - avg_with_zeros_removed).

■ **Tables:** EMPLOYEES(Name,Salary)

■ **Solution:**

```
SELECT CEIL(AVG(Salary)-AVG(CAST(REPLACE(CAST(Salary AS CHAR),'0','') AS UNSIGNED)))  
FROM EMPLOYEES;
```

■ **PRO TIP**

REPLACE removes ALL zeros. CAST back to number. CEIL rounds up the error.

#33 Top Earners

GROUP BY expression

● EASY

■ **Problem:** Max total earnings (salary*months) and count of employees.

■ **Solution:**

```
SELECT salary*months AS earnings, COUNT(*) FROM Employee  
GROUP BY earnings ORDER BY earnings DESC LIMIT 1;
```

■ **INTERVIEW TRICK**

GROUP BY computed alias + ORDER DESC + LIMIT 1 is faster than nested MAX subquery.

#34 Weather Obs. Station 2

SUM ROUND

● EASY

■ **Problem:** Sum LAT_N and sum LONG_W, each rounded 2dp.

■ **Solution:**

```
SELECT ROUND(SUM(LAT_N),2), ROUND(SUM(LONG_W),2) FROM STATION;
```

#35 Weather Obs. Station 13

SUM TRUNCATE range

● EASY

■ **Problem:** Sum LAT_N where $38.788 < \text{LAT_N} < 137.2345$, truncated 4dp.

■ **Solution:**

```
SELECT TRUNCATE(SUM(LAT_N),4) FROM STATION WHERE LAT_N>38.7880 AND LAT_N<137.2345;
```

■■ WATCH OUT

TRUNCATE chops. ROUND may round up. Use exactly what the question specifies.

#36 Weather Obs. Station 14

MAX WHERE

● EASY

■ **Problem:** Greatest LAT_N < 137.2345 , truncated 4dp.

■ **Solution:**

```
SELECT TRUNCATE(MAX(LAT_N),4) FROM STATION WHERE LAT_N<137.2345;
```

#37 Weather Obs. Station 15

ORDER LIMIT

● EASY

■ **Problem:** LONG_W for greatest LAT_N < 137.2345 , rounded 4dp.

■ **Solution:**

```
SELECT ROUND(LONG_W,4) FROM STATION WHERE LAT_N<137.2345 ORDER BY LAT_N DESC LIMIT 1;
```

#38 Weather Obs. Station 16

MIN WHERE

● EASY

■ **Problem:** Smallest LAT_N > 38.7780 , rounded 4dp.

■ **Solution:**

```
SELECT ROUND(MIN(LAT_N),4) FROM STATION WHERE LAT_N>38.7780;
```

#39 Weather Obs. Station 17

ORDER ASC LIMIT

● EASY

■ **Problem:** LONG_W for smallest LAT_N > 38.7780 , rounded 4dp.

■ Solution:

```
SELECT ROUND(LONG_W,4) FROM STATION WHERE LAT_N>38.7780 ORDER BY LAT_N ASC LIMIT 1;
```

#40 Weather Obs. Station 18

Manhattan Distance

◆ MEDIUM

■ Problem: Manhattan distance between P1 and P2, rounded 4dp.

■ Solution:

```
SELECT ROUND(ABS(MAX(LAT_N)-MIN(LAT_N))+ABS(MAX(LONG_W)-MIN(LONG_W)),4) FROM STATION;
```

■ MEMORY TRICK

Manhattan= $|x_1-x_2|+|y_1-y_2|$ (city blocks). Euclidean= $\sqrt{\Delta x^2+\Delta y^2}$ (straight line). Next question is Euclidean!

#41 Weather Obs. Station 19

Euclidean Distance

◆ MEDIUM

■ Problem: Euclidean distance between P1 and P2, rounded 4dp.

■ Solution:

```
SELECT ROUND(SQRT(POW(MAX(LAT_N)-MIN(LAT_N),2)+POW(MAX(LONG_W)-MIN(LONG_W),2)),4)
FROM STATION;
```

■ MEMORY TRICK

SQRT=square root. POW(x,2)= x^2 . Pythagorean theorem: $c=\sqrt{a^2+b^2}$.

#42 Weather Obs. Station 20

MEDIAN

◆ MEDIUM

-- WRONG: integer
division gives
wrong median,
WHERE rn =
COUNT(*)/2 -- 6/2=3,
picks only one row
for even count

■ Problem: Median LAT_N, rounded 4dp.

■ Solution:

```
WITH ordered AS (
  SELECT LAT_N, ROW_NUMBER() OVER (ORDER BY LAT_N) rn, COUNT(*) OVER() total FROM STATION
)
SELECT ROUND(AVG(LAT_N),4) FROM ordered
WHERE rn IN (FLOOR((total+1)/2.0), CEIL((total+1)/2.0));
```

■ KEY PATTERN

ODD count: FLOOR=CEIL → one middle row. EVEN count: FLOOR≠CEIL → two middle rows → AVG them.

■ WATCH OUT

Always /2.0 not /2. MySQL: 5/2=2 (integer). 5/2.0=2.5 (float). Wrong division = wrong median!

#43 Asian Population

INNER JOIN SUM

● EASY

■ **Problem:** Total population of Asian cities.

■ **Tables:** CITY.CountryCode→COUNTRY.Code

■ **Solution:**

```
SELECT SUM(ci.Population) FROM CITY ci
JOIN COUNTRY co ON ci.CountryCode=co.Code
WHERE co.Continent='Asia';
```

#44 African Cities

INNER JOIN

● EASY

■ **Problem:** Names of African cities.

■ **Solution:**

```
SELECT ci.Name FROM CITY ci
JOIN COUNTRY co ON ci.CountryCode=co.Code
WHERE co.Continent='Africa';
```

#45 Avg Population Each Continent

JOIN GROUP BY FLOOR

● EASY

■ **Problem:** Continent name and FLOOR(avg city population).

■ **Solution:**

```
SELECT co.Continent, FLOOR(AVG(ci.Population))
FROM CITY ci JOIN COUNTRY co ON ci.CountryCode=co.Code
GROUP BY co.Continent;
```

■■ WATCH OUT

FLOOR(AVG()) ≠ AVG(FLOOR()). FLOOR applied AFTER averaging.

#46 The Report

JOIN BETWEEN + CASE

◆ MEDIUM

■ **Problem:** NULL for name if grade<8. Order by grade DESC, name/marks.

■ **Tables:** Students(Name,Marks), Grades(Grade,Min_Mark,Max_Mark)

■ **Solution:**

```
SELECT CASE WHEN g.Grade>=8 THEN s.Name ELSE NULL END AS Name, g.Grade, s.Marks
FROM Students s JOIN Grades g ON s.Marks BETWEEN g.Min_Mark AND g.Max_Mark
ORDER BY g.Grade DESC,
CASE WHEN g.Grade>=8 THEN s.Name END ASC,
CASE WHEN g.Grade<8 THEN s.Marks END ASC;
```

■ KEY PATTERN

BETWEEN in JOIN = range join. Two CASE expressions in ORDER BY handle different sort for grade≥8 vs grade<8.

#47 Top Competitors

Multi-JOIN HAVING

◆ MEDIUM

■ **Problem:** Hackers who scored full marks on >1 challenge.

■ **Tables:** Hackers, Submissions, Challenges, Difficulty

■ **Solution:**

```
SELECT h.hacker_id, h.name
FROM Hackers h
JOIN Submissions s ON h.hacker_id=s.hacker_id
JOIN Challenges c ON s.challenge_id=c.challenge_id
JOIN Difficulty d ON c.difficulty_level=d.difficulty_level
WHERE s.score=d.score
GROUP BY h.hacker_id, h.name
HAVING COUNT(*)>1
ORDER BY COUNT(*) DESC, h.hacker_id;
```

■ PRO TIP

WHERE full score first, then HAVING for multiple. Early filtering = less work for GROUP BY.

#48 Ollivander's Inventory

JOIN min-per-group

◆ MEDIUM

■ **Problem:** Non-evil wands with min coins per (power,age) pair.

■ **Tables:** Wands, Wands_Property

■ **Solution:**

```
WITH mc AS (
  SELECT w.power, wp.age, MIN(w.coins_needed) min_c
  FROM Wands w JOIN Wands_Property wp ON w.code=wp.code
  WHERE wp.is_evil=0 GROUP BY w.power, wp.age
)
SELECT w.id, wp.age, w.coins_needed, w.power
FROM Wands w JOIN Wands_Property wp ON w.code=wp.code
JOIN mc ON w.power=mc.power AND wp.age=mc.age AND w.coins_needed=mc.min_c
WHERE wp.is_evil=0 ORDER BY w.power DESC, wp.age DESC;
```

■ INTERVIEW TRICK

Pre-aggregate min coins in CTE then join back — avoids N correlated subquery executions.

#49 Challenges

CTE + filter logic

◆ MEDIUM

■ **Problem:** Include if count=max OR count is unique.

■ **Tables:** Hackers, Challenges

■ **Solution:**

```
WITH cnt AS (
  SELECT h.hacker_id, h.name, COUNT(c.challenge_id) AS total
  FROM Hackers h JOIN Challenges c USING(hacker_id) GROUP BY h.hacker_id, h.name
),
mx AS (SELECT MAX(total) max_total FROM cnt)
SELECT hacker_id, name, total FROM cnt CROSS JOIN mx
WHERE total=max_total
      OR total NOT IN (SELECT total FROM cnt GROUP BY total HAVING COUNT(*)>1)
ORDER BY total DESC, hacker_id;
```

■ KEY PATTERN

INCLUDE if max (always show top) OR unique count. EXCLUDE only if: not max AND count shared by others.

#50 Contest Leaderboard

CTE MAX per challenge

◆ MEDIUM

-- WRONG: sums all attempts, not best per challenge,
SELECT hacker_id, SUM(score) FROM Submissions GROUP BY hacker_id

■ **Problem:** Sum of best scores per challenge per hacker.

■ **Tables:** Hackers, Submissions

■ **Solution:**

```
WITH best AS (
  SELECT hacker_id, challenge_id, MAX(score) best_score
  FROM Submissions GROUP BY hacker_id, challenge_id
)
SELECT h.hacker_id, h.name, SUM(b.best_score) total
FROM Hackers h JOIN best b USING(hacker_id)
GROUP BY h.hacker_id, h.name HAVING total>0
ORDER BY total DESC, h.hacker_id;
```

■■ WATCH OUT

Two-step aggregation: FIRST MAX per (hacker,challenge), THEN SUM of those maxes. Never single GROUP BY!

#51 SQL Project Planning

Gap & Island Dates

▲ HARD

■ **Problem:** Consecutive tasks = same project. Find start/end dates.

■ **Tables:** Projects(Task_ID,Start_Date,End_Date)

■ **Solution:**

```
WITH starts AS (SELECT Start_Date FROM Projects WHERE Start_Date NOT IN (SELECT End_Date FROM Projects)),
ends AS (SELECT End_Date FROM Projects WHERE End_Date NOT IN (SELECT Start_Date FROM Projects))
SELECT s.Start_Date, MIN(e.End_Date) FROM starts s JOIN ends e ON e.End_Date>s.Start_Date
GROUP BY s.Start_Date
ORDER BY DATEDIFF(MIN(e.End_Date),s.Start_Date) ASC, s.Start_Date ASC;
```


■ KEY PATTERN

A date that is NOT someone else's End_Date = new project start. This is the Gap & Island insight.

#52 Placements

Multi-JOIN salary comparison

◆ MEDIUM

■ **Problem:** Students whose friend got higher salary offer.

■ **Tables:** Students, Friends, Packages

■ **Solution:**

```
SELECT s.Name FROM Students s
JOIN Friends f ON s.ID=f.ID
JOIN Packages p1 ON s.ID=p1.ID
JOIN Packages p2 ON f.Friend_ID=p2.ID
WHERE p2.Salary>p1.Salary ORDER BY p2.Salary;
```

■ PRO TIP

p1=student salary, p2=friend salary. Alias same table twice.

#53 Symmetric Pairs

Self JOIN math

◆ MEDIUM

■ **Problem:** Output symmetric pairs where (X1,Y1)■(X2,Y2).

■ **Tables:** Functions(X,Y)

■ **Solution:**

```
SELECT DISTINCT f1.X, f1.Y FROM Functions f1
JOIN Functions f2 ON f1.X=f2.Y AND f1.Y=f2.X
WHERE f1.X<=f1.Y
AND (f1.X<f1.Y OR (SELECT COUNT(*) FROM Functions WHERE X=f1.X AND Y=f1.Y)>1)
ORDER BY f1.X;
```

■ KEY PATTERN

f1.X<=f1.Y prevents outputting (2,3) and (3,2). Self-symmetric (X=Y) pairs need count>1.

#54 Interviews

Pre-aggregation multi-JOIN

▲ HARD

-- WRONG: joining
raw stats without
pre-aggregating,
LEFT JOIN
Submission_Stats
ss ON ch.challenge
_id=ss.challenge_id
, -- 5 stat rows × 3
view rows = 15 rows
per challenge →
overcounted!

■ **Problem:** Sum of submissions/views per contest. Exclude if all zeros.

■ **Tables:** Contests, Colleges, Challenges, Submission_Stats, View_Stats

■ **Solution:**

```
SELECT co.contest_id, co.hacker_id, co.name,
       SUM(IFNULL(ss.ts,0)),SUM(IFNULL(ss.tas,0)),SUM(IFNULL(vs.tv,0)),SUM(IFNULL(vs.tuv,0))
FROM Contests co JOIN Colleges cl ON co.contest_id=cl.contest_id
JOIN Challenges ch ON cl.college_id=ch.college_id
LEFT JOIN (SELECT challenge_id,SUM(total_submissions) ts,SUM(total_accepted_submissions) tas
           FROM Submission_Stats GROUP BY challenge_id) ss ON ch.challenge_id=ss.challenge_id
LEFT JOIN (SELECT challenge_id,SUM(total_views) tv,SUM(total_unique_views) tuv
           FROM View_Stats GROUP BY challenge_id) vs ON ch.challenge_id=vs.challenge_id
GROUP BY co.contest_id,co.hacker_id,co.name
HAVING SUM(IFNULL(ss.ts,0))+SUM(IFNULL(ss.tas,0))+SUM(IFNULL(vs.tv,0))+SUM(IFNULL(vs.tuv,0))>0
ORDER BY co.contest_id;
```

■ WATCH OUT

PRE-AGGREGATE stats per challenge_id FIRST. Raw multi-join creates row explosion → wrong counts!

#55 15 Days of Learning SQL

Window + Streak

▲ HARD

■ **Problem:** Daily streak count + top submitter per day.

■ **Tables:** Hackers, Submissions

■ **Solution:**

```
WITH daily AS (SELECT submission_date, hacker_id, COUNT(*) sub_cnt
                FROM Submissions GROUP BY submission_date, hacker_id),
streak AS (SELECT submission_date, COUNT(DISTINCT hacker_id) streak_cnt FROM daily d1
           WHERE (SELECT COUNT(DISTINCT submission_date) FROM daily d2
                  WHERE d2.hacker_id=d1.hacker_id AND d2.submission_date<=d1.submission_date)
           = (SELECT COUNT(DISTINCT submission_date) FROM daily d3
              WHERE d3.submission_date<=d1.submission_date)
           GROUP BY submission_date),
top_h AS (SELECT submission_date, hacker_id,
                ROW_NUMBER() OVER (PARTITION BY submission_date ORDER BY sub_cnt DESC, hacker_i
d) rn
          FROM daily)
SELECT s.submission_date, s.streak_cnt, t.hacker_id, h.name
FROM streak s JOIN top_h t ON s.submission_date=t.submission_date AND t.rn=1
JOIN Hackers h ON t.hacker_id=h.hacker_id ORDER BY s.submission_date;
```

■ KEY PATTERN

Streak: hacker's submission days up to D = total contest days up to D. Always break into 3 CTEs!

#56 Draw Triangle 1

Recursive CTE

● EASY

■ **Problem:** 20 rows descending: 20 asterisks to 1.

■ **Solution:**

```
WITH RECURSIVE t(n) AS (SELECT 20 UNION ALL SELECT n-1 FROM t WHERE n>1)
SELECT REPEAT('* ',n) FROM t;
```

■ WATCH OUT

WHERE n>1 is the termination condition. Without it → infinite loop!

#57 Draw Triangle 2

Recursive CTE

● EASY

■ **Problem:** 20 rows ascending: 1 asterisk to 20.

■ **Solution:**

```
WITH RECURSIVE t(n) AS (SELECT 1 UNION ALL SELECT n+1 FROM t WHERE n<20)
SELECT REPEAT('* ',n) FROM t;
```

#58 Print Prime Numbers

Recursive CTE + Trial Division

◆ MEDIUM

■ **Problem:** All primes ≤ 1000 as '&'-delimited string.

■ **Solution:**

```
WITH RECURSIVE nums(n) AS (SELECT 2 UNION ALL SELECT n+1 FROM nums WHERE n<1000),
primes AS (SELECT n FROM nums p WHERE NOT EXISTS (
    SELECT 1 FROM nums d WHERE d.n BETWEEN 2 AND FLOOR(SQRT(p.n)) AND p.n%d.n=0))
SELECT GROUP_CONCAT(n ORDER BY n SEPARATOR '&') FROM primes;
```

■ KEY PATTERN

Only check divisors up to $\sqrt{n}$. If no divisor up to $\sqrt{n} \rightarrow$ prime. Reduces $O(n)$ to $O(\sqrt{n})$ per number.

⇒ ■ **PRACTICE:** Write HackerRank Q10 (Station 5) from scratch without looking. | Write the Occupations Pivot (Q23) from scratch — this appears in real interviews.

SECTION 7

LeetCode

SQL 50

All 50 official problems — the world's most-used SQL interview prep set

Top-N Per Group and Running Total Diagrams:

Top-N Per Group — The Most Frequently Asked Window Function Pattern

Name	Dept	Salary	DENSE_RANK() per Dept	Keep? (rn<=3)
Alice	Sales	90,000	rn=1	■ YES
Bob	Sales	80,000	rn=2	■ YES
Carol	Sales	70,000	rn=3	■ YES
Dave	Sales	60,000	rn=4 ■	■ NO
Eve	Eng	95,000	rn=1	■ YES
Frank	Eng	85,000	rn=2	■ YES
Grace	Eng	75,000	rn=3	■ YES
Hank	Eng	65,000	rn=4 ■	■ NO

Running Total — SUM() OVER (ORDER BY date ROWS UNBOUNDED PRECEDING)

Date	Revenue	Running Total
Jan 1	1,000	1,000
Jan 2	2,500	3,500
Jan 3	500	4,000
Jan 4	3,000	7,000
Jan 5	1,500	8,500

#59 Recyclable and Low Fat Products

LeetCode SELECT

● EASY

■ **Problem:** Find products where low_fats='Y' AND recyclable='Y'.

■ **Tables:** Products(product_id,low_fats,recyclable)

■ **Solution:**

```
SELECT product_id FROM Products WHERE low_fats='Y' AND recyclable='Y';
```

■ PRO TIP

Simple AND filter. Know it in 10 seconds.

#60 Find Customer Referee

LeetCode NULL

● EASY

Amazon

■ **Problem:** Customers NOT referred by id=2. Include NULL referee_id.

■ **Tables:** Customer(id,name,referee_id)

■ Common Mistake:

```
-- WRONG: misses NULL referee customers
SELECT name FROM Customer WHERE referee_id != 2;
```

■ Solution:

```
SELECT name FROM Customer WHERE referee_id!=2 OR referee_id IS NULL;
```

■■ WATCH OUT

NULL TRAP: referee_id!=2 is NULL when referee_id IS NULL. Must add OR IS NULL.

#61 Big Countries

LeetCode OR

● EASY

■ **Problem:** Country is big if area $\geq$ 3,000,000 OR population $\geq$ 25,000,000.

■ **Tables:** World(name,continent,area,population)

■ Solution:

```
SELECT name,population,area FROM world WHERE area>=3000000 OR population>=25000000;
```

#62 Article Views I

LeetCode self-filter

● EASY

Meta

■ **Problem:** Authors who viewed their own articles (author_id=viewer_id).

■ **Tables:** Views(article_id,author_id,viewer_id)

■ Solution:

```
SELECT DISTINCT author_id AS id FROM Views WHERE author_id=viewer_id ORDER BY id;
```

■ KEY PATTERN

Self-referential filter: author_id = viewer_id. Common in social media data.

#63 Invalid Tweets

LeetCode LENGTH

● EASY

■ **Problem:** Tweets with content length > 15.

■ **Tables:** Tweets(tweet_id,content)

■ Solution:

```
SELECT tweet_id FROM Tweets WHERE LENGTH(content)>15;
```

■■ WATCH OUT

LENGTH() $\neq$ bytes. CHAR_LENGTH() $\neq$ characters. Use CHAR_LENGTH for Unicode/emoji.

#64 Replace Employee ID

LeetCode LEFT JOIN

● EASY

■ **Problem:** Show unique_id for each employee. NULL if no mapping.

■ **Tables:** Employees(id,name), EmployeeUNI(id,unique_id)

■ **Solution:**

```
SELECT eu.unique_id, e.name FROM Employees e LEFT JOIN EmployeeUNI eu ON e.id=eu.id;
```

■ **KEY PATTERN**

LEFT JOIN: keep ALL from left, NULL from right if no match. Most common JOIN in analytics.

#65 Product Sales Analysis I

LeetCode INNER JOIN

● EASY

■ **Problem:** Report product_name, year, price for all sales.

■ **Tables:** Sales,Product tables

■ **Solution:**

```
SELECT p.product_name,s.year,s.price FROM Sales s JOIN Product p ON s.product_id=p.product_id;
```

#66 Customer No Transactions

LeetCode Anti-JOIN

● EASY

Amazon

■ **Problem:** Customers with visits but no transactions. Count such visits.

■ **Tables:** Visits,Transactions

■ **Solution:**

```
SELECT v.customer_id, COUNT(v.visit_id) AS count_no_trans
FROM Visits v LEFT JOIN Transactions t ON v.visit_id=t.visit_id
WHERE t.transaction_id IS NULL GROUP BY v.customer_id;
```

■ **KEY PATTERN**

ANTI-JOIN: LEFT JOIN + WHERE right.id IS NULL = rows with no match. Memorize this pattern!

#67 Rising Temperature

LeetCode Self JOIN Date

● EASY

Google

■ **Problem:** Dates where temperature was higher than previous day.

■ **Tables:** Weather(id,recordDate,temperature)

■ **Solution:**

```
SELECT w1.id FROM Weather w1 JOIN Weather w2 ON DATEDIFF(w1.recordDate,w2.recordDate)=1
WHERE w1.temperature>w2.temperature;
```

■ **INTERVIEW TRICK**

DATEDIFF(later,earlier)=1 ensures exactly previous day. Modern: LAG(temp) OVER (ORDER BY date).

#68 Average Process Time

LeetCode Conditional Agg

● EASY

■ **Problem:** Average process time per machine, rounded 3dp.

■ **Tables:** Activity(machine_id,process_id,activity_type,timestamp)

■ **Solution:**

```
SELECT machine_id,ROUND(AVG(end_time-start_time),3) AS processing_time
FROM (SELECT machine_id,process_id,
      MAX(CASE WHEN activity_type='end' THEN timestamp END) end_time,
      MAX(CASE WHEN activity_type='start' THEN timestamp END) start_time
      FROM Activity GROUP BY machine_id,process_id) t
GROUP BY machine_id;
```

■ **KEY PATTERN**

Pivot start/end using MAX(CASE WHEN). Subtract to get duration. Average across processes.

#69 Employee Bonus

LeetCode LEFT JOIN NULL

● EASY

■ **Problem:** Employees with bonus < 1000 OR no bonus record.

■ **Tables:** Employee,Bonus

■ **Solution:**

```
SELECT e.name,b.bonus FROM Employee e LEFT JOIN Bonus b ON e.empId=b.empId
WHERE b.bonus<1000 OR b.bonus IS NULL;
```

■ **WATCH OUT**

OR IS NULL required — employees without bonus row (NULL from LEFT JOIN) must be included.

#70 Students and Examinations

LeetCode CROSS + LEFT JOIN

● EASY

■ **Problem:** Count exam attendances per student per subject. Include zeros.

■ **Tables:** Students,Subjects,Examinations

■ **Solution:**

```
SELECT s.student_id,s.student_name,sub.subject_name,COUNT(e.subject_name) attended_exams
FROM Students s CROSS JOIN Subjects sub
LEFT JOIN Examinations e ON s.student_id=e.student_id AND sub.subject_name=e.subject_name
GROUP BY s.student_id,s.student_name,sub.subject_name ORDER BY s.student_id,sub.subject_name;
```

■ **KEY PATTERN**

CROSS JOIN generates all (student×subject) combos for zero-attendance rows.

#71 Managers with 5+ Reports

LeetCode Self JOIN HAVING

◆ MEDIUM

Google

■ **Problem:** Managers with at least 5 direct reports.

■ **Tables:** Employee(id,name,managerId)

■ **Solution:**

```
SELECT e1.name FROM Employee e1 JOIN Employee e2 ON e1.id=e2.managerId
GROUP BY e1.id,e1.name HAVING COUNT(*)>=5;
```

■ KEY PATTERN

Self-join: e1=managers, e2=reports. GROUP BY manager, HAVING ≥5.

#72 Confirmation Rate

LeetCode AVG CASE ratio

◆ MEDIUM

■ **Problem:** Confirmation rate per user = confirmed/total. Round 2dp.

■ **Tables:** Signups, Confirmations

■ **Solution:**

```
SELECT s.user_id, ROUND(AVG(CASE WHEN c.action='confirmed' THEN 1.0 ELSE 0.0 END), 2) confirmation_rate
FROM Signups s LEFT JOIN Confirmations c ON s.user_id=c.user_id GROUP BY s.user_id;
```

■ MEMORY TRICK

AVG(CASE WHEN cond THEN 1.0 ELSE 0.0 END) = ratio/percentage. Universal pattern — memorize it!

#73 Not Boring Movies

LeetCode MOD filter

● EASY

■ **Problem:** Movies: odd id AND description!='boring'. Sort by rating DESC.

■ **Tables:** Cinema(id, movie, description, rating)

■ **Solution:**

```
SELECT * FROM Cinema WHERE id%2=1 AND description!='boring' ORDER BY rating DESC;
```

#74 Average Selling Price

LeetCode Weighted Avg

● EASY

■ **Problem:** Weighted avg selling price per product. Round 2dp.

■ **Tables:** Prices, UnitsSold

■ **Solution:**

```
SELECT p.product_id, ROUND(SUM(p.price*u.units)/SUM(u.units), 2) average_price
FROM Prices p LEFT JOIN UnitsSold u ON p.product_id=u.product_id
AND u.purchase_date BETWEEN p.start_date AND p.end_date GROUP BY p.product_id;
```

■ KEY PATTERN

Weighted avg = SUM(price×units)/SUM(units). NOT AVG(price).

#75 Project Employees I

LeetCode JOIN AVG

● EASY

■ **Problem:** Avg experience years per project, rounded 2dp.

■ **Tables:** Project,Employee

■ **Solution:**

```
SELECT p.project_id,ROUND(AVG(e.experience_years),2) average_years
FROM Project p JOIN Employee e ON p.employee_id=e.employee_id GROUP BY p.project_id;
```

#76 Contest Percentage

LeetCode percentage

● EASY

■ **Problem:** % of users registered per contest. Sort % DESC, contest_id ASC.

■ **Tables:** Users,Register

■ **Solution:**

```
SELECT contest_id,ROUND(COUNT(DISTINCT user_id)*100.0/(SELECT COUNT(*) FROM Users),2) AS percent
age
FROM Register GROUP BY contest_id ORDER BY percentage DESC, contest_id ASC;
```

■■ WATCH OUT

×100.0 not ×100 — avoids integer division. Always use .0 for percentages!

#77 Query Quality

LeetCode multi metrics

● EASY

■ **Problem:** Quality (avg rating/position) and poor_query_percentage per query.

■ **Tables:** Queries

■ **Solution:**

```
SELECT query_name,ROUND(AVG(rating/position),2) quality,
ROUND(AVG(CASE WHEN rating<3 THEN 100.0 ELSE 0 END),2) poor_query_percentage
FROM Queries WHERE query_name IS NOT NULL GROUP BY query_name;
```

#78 Monthly Transactions I

LeetCode DATE_FORMAT SUM

◆ MEDIUM

Amazon

■ **Problem:** Per month+country: total txns, approved count, amounts.

■ **Tables:** Transactions

■ **Solution:**

```
SELECT DATE_FORMAT(trans_date,'%Y-%m') month,country,COUNT(*) trans_count,
SUM(CASE WHEN state='approved' THEN 1 ELSE 0 END) approved_count,
SUM(amount) trans_total_amount,
SUM(CASE WHEN state='approved' THEN amount ELSE 0 END) approved_total_amount
FROM Transactions GROUP BY month,country;
```

■ MEMORY TRICK

DATE_FORMAT('%Y-%m') = 'YYYY-MM' month grouping. SUM(CASE WHEN) = conditional count.

#79 Immediate Food Delivery II

LeetCode first order

◆ MEDIUM

■ **Problem:** % customers whose FIRST order was immediate. Round 2dp.

■ **Tables:** Delivery

■ **Solution:**

```
WITH fo AS (SELECT customer_id,MIN(order_date) first_date FROM Delivery GROUP BY customer_id)
SELECT ROUND(100.0*SUM(CASE WHEN d.order_date=d.customer_pref_delivery_date THEN 1 ELSE 0 END)/COUNT(*),2)
AS immediate_percentage
FROM Delivery d JOIN fo ON d.customer_id=fo.customer_id AND d.order_date=fo.first_date;
```

■ **KEY PATTERN**

Find first order date per customer (CTE), then check if that order was immediate.

#80 Game Play Retention

LeetCode D1 retention

◆ MEDIUM

Meta

■ **Problem:** Fraction of players who logged in the day after first login.

■ **Tables:** Activity

■ **Solution:**

```
WITH fl AS (SELECT player_id,MIN(event_date) first_date FROM Activity GROUP BY player_id)
SELECT ROUND(COUNT(DISTINCT a.player_id)/(SELECT COUNT(DISTINCT player_id) FROM Activity),2) fraction
FROM Activity a JOIN fl ON a.player_id=fl.player_id
AND a.event_date=DATE_ADD(fl.first_date,INTERVAL 1 DAY);
```

■ **KEY PATTERN**

Day-1 retention = % who came back next day. DATE_ADD adds exactly 1 day.

#81 Unique Subjects Per Teacher

LeetCode COUNT DISTINCT

● EASY

■ **Problem:** Count unique subjects each teacher teaches.

■ **Tables:** Teacher

■ **Solution:**

```
SELECT teacher_id,COUNT(DISTINCT subject_id) cnt FROM Teacher GROUP BY teacher_id;
```

#82 Daily Active Users 30 Days

LeetCode date range

● EASY

■ **Problem:** Daily active users for 30 days ending 2019-07-27.

■ **Tables:** Activity

■ **Solution:**

```
SELECT activity_date day,COUNT(DISTINCT user_id) active_users FROM Activity
WHERE activity_date BETWEEN DATE_SUB('2019-07-27',INTERVAL 29 DAY) AND '2019-07-27'
GROUP BY activity_date;
```

■ WATCH OUT

BETWEEN inclusive. 30 days = today + 29 days before = INTERVAL 29 DAY (not 30).

#83 Product First Year

LeetCode first year

◆ MEDIUM

■ **Problem:** Each product's first year sales (year,quantity,price).

■ **Tables:** Sales

■ **Solution:**

```
WITH fy AS (SELECT product_id,MIN(year) first_year FROM Sales GROUP BY product_id)
SELECT s.product_id,s.year first_year,s.quantity,s.price
FROM Sales s JOIN fy ON s.product_id=fy.product_id AND s.year=fy.first_year;
```

#84 Classes 5+ Students

LeetCode HAVING

● EASY

■ **Problem:** Classes with at least 5 students.

■ **Tables:** Courses(student,class)

■ **Solution:**

```
SELECT class FROM Courses GROUP BY class HAVING COUNT(student)>=5;
```

#85 Followers Count

LeetCode COUNT

● EASY

■ **Problem:** Follower count per user, sorted by user_id.

■ **Tables:** Followers

■ **Solution:**

```
SELECT user_id,COUNT(follower_id) followers_count FROM Followers GROUP BY user_id ORDER BY user_id;
```

#86 Biggest Single Number

LeetCode MAX singles

● EASY

■ **Problem:** Largest number appearing exactly once. NULL if none.

■ **Tables:** MyNumbers(num)

■ **Solution:**

```
SELECT MAX(num) AS num FROM MyNumbers GROUP BY num HAVING COUNT(*)=1;
```

■ PRO TIP

MAX on empty set = NULL automatically. Perfect for 'return NULL if not exists'.

#87 Customers Bought All Products

LeetCode relational division

◆ MEDIUM

■ **Problem:** Customers who bought ALL products.

■ **Tables:** Customer, Product

■ **Solution:**

```
SELECT customer_id FROM Customer
GROUP BY customer_id HAVING COUNT(DISTINCT product_key)=(SELECT COUNT(*) FROM Product);
```

■ KEY PATTERN

RELATIONAL DIVISION: count distinct Ys per X = total Y count. Classic pattern!

#88 Employee Reports

LeetCode self JOIN

● EASY

■ **Problem:** Manager: count reports and round average age.

■ **Tables:** Employees

■ **Solution:**

```
SELECT e1.employee_id, e1.name, COUNT(e2.employee_id) reports_count, ROUND(AVG(e2.age)) average_age
FROM Employees e1 JOIN Employees e2 ON e1.employee_id=e2.reports_to
GROUP BY e1.employee_id, e1.name ORDER BY e1.employee_id;
```

#89 Primary Department

LeetCode UNION

● EASY

■ **Problem:** Each employee's primary department.

■ **Tables:** Employee(employee_id, department_id, primary_flag)

■ **Solution:**

```
SELECT employee_id, department_id FROM Employee WHERE primary_flag='Y'
UNION
SELECT employee_id, department_id FROM Employee GROUP BY employee_id HAVING COUNT(*)=1;
```

■ KEY PATTERN

Two cases: explicit flag='Y' OR only one department. UNION deduplicates.

#90 Triangle Judgement

LeetCode CASE

● EASY

■ **Problem:** Yes/No if (x,y,z) can form a triangle.

■ **Tables:** Triangle(x,y,z)

■ **Solution:**

```
SELECT x,y,z,CASE WHEN x+y>z AND x+z>y AND y+z>x THEN 'Yes' ELSE 'No' END AS triangle
FROM Triangle;
```

#91 Consecutive Numbers

LeetCode Gap Island

◆ MEDIUM

Google,Amazon

■ **Problem:** Numbers appearing 3+ times consecutively.

■ **Tables:** Logs(id,num)

■ **Common Mistake:**

```
-- WRONG approach (works but O(n^2))
SELECT DISTINCT l1.num FROM Logs l1
JOIN Logs l2 ON l2.id=l1.id+1 AND l2.num=l1.num
JOIN Logs l3 ON l3.id=l1.id+2 AND l3.num=l1.num;
```

■ **Solution:**

```
WITH g AS (SELECT num, id-ROW_NUMBER() OVER (PARTITION BY num ORDER BY id) grp FROM Logs)
SELECT DISTINCT num ConsecutiveNums FROM g GROUP BY num,grp HAVING COUNT(*)>=3;
```

■ KEY PATTERN

Gap & Island: subtract ROW_NUMBER per value group from id. Consecutive same values = same grp.

#92 Product Price at Date

LeetCode latest-before-date

◆ MEDIUM

■ **Problem:** Price of each product on 2019-08-16. Default=10 if no prior changes.

■ **Tables:** Products

■ **Solution:**

```
WITH latest AS (
  SELECT product_id,new_price,ROW_NUMBER() OVER (PARTITION BY product_id ORDER BY change_date DESC) rn
  FROM Products WHERE change_date<='2019-08-16'
)
SELECT p.product_id,IFNULL(l.new_price,10) price
FROM (SELECT DISTINCT product_id FROM Products) p
LEFT JOIN latest l ON p.product_id=l.product_id AND l.rn=1;
```

■ KEY PATTERN

ROW_NUMBER DESC on date = rn 1 for most recent change. IFNULL handles products with no prior change.

#93 Last Person on Bus

LeetCode running SUM

◆ MEDIUM

■ **Problem:** Last person who can board without exceeding 1000kg.

■ **Tables:** Queue(person_id,person_name,weight,turn)

■ **Solution:**

```
WITH r AS (SELECT person_name,SUM(weight) OVER (ORDER BY turn ROWS UNBOUNDED PRECEDING) cw FROM Queue)
SELECT person_name FROM r WHERE cw<=1000 ORDER BY cw DESC LIMIT 1;
```

■ KEY PATTERN

Running SUM window, filter ≤ 1000 , take highest cumulative = closest to limit.

#94 Count Salary Categories

LeetCode UNION ALL categories

◆ MEDIUM

■ **Problem:** Count accounts in Low/Average/High salary buckets.

■ **Tables:** Accounts(income)

■ **Solution:**

```
SELECT 'Low Salary',SUM(CASE WHEN income<20000 THEN 1 ELSE 0 END) FROM Accounts UNION ALL
SELECT 'Average Salary',SUM(CASE WHEN income BETWEEN 20000 AND 50000 THEN 1 ELSE 0 END) FROM Accounts UNION ALL
SELECT 'High Salary',SUM(CASE WHEN income>50000 THEN 1 ELSE 0 END) FROM Accounts;
```

■ WATCH OUT

UNION ALL of hard-coded categories ensures all 3 rows appear even if count=0.

#95 Employees Without Manager Left

LeetCode NOT IN

● EASY

■ **Problem:** Employees (salary<30000) whose manager has left.

■ **Tables:** Employees(employee_id,name,manager_id,salary)

■ **Solution:**

```
SELECT employee_id FROM Employees
WHERE salary<30000 AND manager_id IS NOT NULL
AND manager_id NOT IN (SELECT employee_id FROM Employees) ORDER BY employee_id;
```

■ PRO TIP

IS NOT NULL check first — top-level managers (NULL) haven't 'left'.

#96 Exchange Seats

LeetCode CASE swap

◆ MEDIUM

■ **Problem:** Swap adjacent students' seats. Last stays if odd count.

■ **Tables:** Seat(id,student)

■ **Solution:**

```
SELECT CASE WHEN MOD(id,2)=1 AND id<(SELECT MAX(id) FROM Seat) THEN id+1
            WHEN MOD(id,2)=0 THEN id-1 ELSE id END AS id, student
FROM Seat ORDER BY id;
```

■ KEY PATTERN

Three cases: odd+not-last→id+1, even→id-1, last odd→stay.

#97 Movie Rating

LeetCode UNION ALL rankings

◆ MEDIUM

■ **Problem:** User who rated most movies; movie with highest avg in Feb 2020.

■ **Tables:** MovieRating, Movies, Users

■ **Solution:**

```
(SELECT u.name results FROM MovieRating mr JOIN Users u ON mr.user_id=u.user_id
GROUP BY mr.user_id ORDER BY COUNT(*) DESC, u.name ASC LIMIT 1)
UNION ALL
(SELECT m.title FROM MovieRating mr JOIN Movies m ON mr.movie_id=m.movie_id
WHERE DATE_FORMAT(created_at, '%Y-%m')='2020-02'
GROUP BY mr.movie_id ORDER BY AVG(rating) DESC, m.title ASC LIMIT 1);
```

■ ■ WATCH OUT

Parentheses allow independent ORDER BY + LIMIT per SELECT before UNION ALL.

#98 Restaurant Growth

LeetCode 7-day rolling

◆ MEDIUM

■ **Problem:** 7-day moving total and average. Only from day 7 onward.

■ **Tables:** Customer(customer_id, name, visited_on, amount)

■ **Solution:**

```
SELECT visited_on,
SUM(SUM(amount)) OVER (ORDER BY visited_on ROWS BETWEEN 6 PRECEDING AND CURRENT ROW) amount,
ROUND(AVG(SUM(amount)) OVER (ORDER BY visited_on ROWS BETWEEN 6 PRECEDING AND CURRENT ROW), 2) av
erage_amount
FROM Customer GROUP BY visited_on
HAVING visited_on >= DATE_ADD(MIN(visited_on) OVER(), INTERVAL 6 DAY);
```

■ KEY PATTERN

ROWS BETWEEN 6 PRECEDING AND CURRENT ROW = 7-row window.

#99 Most Friends

LeetCode UNION ALL both sides

◆ MEDIUM

Meta

■ **Problem:** Person with most total friends (requester + acceptor).

■ **Tables:** RequestAccepted

■ **Solution:**

```
WITH af AS (SELECT requester_id id FROM RequestAccepted UNION ALL SELECT acceptor_id FROM Reques
tAccepted)
SELECT id, COUNT(*) num FROM af GROUP BY id ORDER BY num DESC LIMIT 1;
```

■ ■ WATCH OUT

Both sides count! UNION ALL keeps duplicates — each friendship counted once per person.

#100 Investments in 2016

LeetCode EXISTS NOT EXISTS

◆ MEDIUM

■ **Problem:** Sum tiv_2016 for policyholders who: share tiv_2015 AND have unique city.

■ **Tables:** Insurance(pid, tiv_2015, tiv_2016, lat, lon)

■ Solution:

```
SELECT ROUND(SUM(tiv_2016),2) AS tiv_2016 FROM Insurance i1
WHERE EXISTS (SELECT 1 FROM Insurance i2 WHERE i2.pid!=i1.pid AND i2.tiv_2015=i1.tiv_2015)
AND NOT EXISTS (SELECT 1 FROM Insurance i2 WHERE i2.pid!=i1.pid AND i2.lat=i1.lat AND i2.lon=i1.lon);
```

■ KEY PATTERN

EXISTS='at least one other'. NOT EXISTS='no other'. Both short-circuit on first match.

#101 Department Top 3 Salaries

LeetCode DENSE_RANK

▲ HARD

Google,Amazon

■ **Problem:** Employees in top 3 unique salary tiers per department.

■ **Tables:** Employee,Department

■ Solution:

```
WITH r AS (SELECT e.name Employee,d.name Department,e.salary Salary,
DENSE_RANK() OVER (PARTITION BY e.departmentId ORDER BY e.salary DESC) rnk
FROM Employee e JOIN Department d ON e.departmentId=d.id)
SELECT Department,Employee,Salary FROM r WHERE rnk<=3;
```

■ KEY PATTERN

DENSE_RANK: ties get same rank, no gaps. PARTITION BY resets per dept.

#102 Fix Names Capitalization

LeetCode string

● EASY

■ **Problem:** First letter uppercase, rest lowercase.

■ **Tables:** Users(user_id,name)

■ Solution:

```
SELECT user_id,CONCAT(UPPER(SUBSTRING(name,1,1)),LOWER(SUBSTRING(name,2))) name FROM Users ORDER
BY user_id;
```

■ MEMORY TRICK

PostgreSQL shortcut: INITCAP(name). MySQL needs CONCAT approach.

#103 Patients With DIAB1

LeetCode LIKE

● EASY

■ **Problem:** Patients with DIAB1 condition code.

■ **Tables:** Patients(patient_id,patient_name,conditions)

■ Solution:

```
SELECT patient_id,patient_name,conditions FROM Patients
WHERE conditions LIKE 'DIAB1%' OR conditions LIKE '% DIAB1%';
```

■ PRO TIP

Space before DIAB1 = word boundary. Prevents matching 'ADIAB1'.

#104 Delete Duplicate Emails

LeetCode DELETE JOIN

● EASY

■ **Problem:** Delete duplicate emails, keep smallest id.

■ **Tables:** Person(id,email)

■ **Common Mistake:**

```
-- WRONG in MySQL: can't reference same table in FROM
DELETE FROM Person WHERE id NOT IN (SELECT MIN(id) FROM Person GROUP BY email);
-- ERROR: can't specify target table 'Person' for update in FROM
```

■ **Solution:**

```
DELETE p1 FROM Person p1 JOIN Person p2 ON p1.email=p2.email AND p1.id>p2.id;
```

■ WATCH OUT

Self-join DELETE: delete higher id. Never DELETE WHERE id NOT IN (same table) in MySQL — error!

#105 Second Highest Salary

LeetCode MAX subquery

◆ MEDIUM

Amazon,Microsoft

■ **Problem:** Second highest distinct salary. NULL if not exists.

■ **Tables:** Employee(id,salary)

■ **Solution:**

```
SELECT MAX(salary) AS SecondHighestSalary
FROM Employee WHERE salary<(SELECT MAX(salary) FROM Employee);
```

■ MEMORY TRICK

MAX on empty set = NULL. Know 3 approaches: subquery MAX, DENSE_RANK, LIMIT OFFSET.

#106 Group Sold Products By Date

LeetCode GROUP_CONCAT

● EASY

■ **Problem:** Per date: count distinct products and sorted names.

■ **Tables:** Activities(sell_date,product)

■ **Solution:**

```
SELECT sell_date,COUNT(DISTINCT product) num_sold,
GROUP_CONCAT(DISTINCT product ORDER BY product SEPARATOR ',') products
FROM Activities GROUP BY sell_date ORDER BY sell_date;
```

■ MEMORY TRICK

GROUP_CONCAT(DISTINCT...ORDER BY...SEPARATOR) = MySQL string aggregation. PostgreSQL: STRING_AGG.

#107 Products Ordered 100+ Feb 2020

LeetCode DATE HAVING

● EASY

■ **Problem:** Products with 100+ units ordered in February 2020.

■ **Tables:** Products,Orders

■ **Solution:**

```
SELECT p.product_name,SUM(o.unit) unit FROM Products p JOIN Orders o ON p.product_id=o.product_id
WHERE DATE_FORMAT(order_date,'%Y-%m')='2020-02' GROUP BY p.product_id,p.product_name HAVING SUM(
o.unit)>=100;
```

#108 Valid Email Regex

LeetCode REGEXP

● EASY

■ **Problem:** Users with valid emails (letter-start prefix, @leetcode.com).

■ **Tables:** Users(user_id,name,mail)

■ **Solution:**

```
SELECT * FROM Users WHERE mail REGEXP '^[A-Za-z][A-Za-z0-9_-.]*@leetcode\.com$';
```

■ MEMORY TRICK

\. = literal dot. ^ = start. \$ = end. [A-Za-z0-9_-.]* = valid prefix chars.

==■ **PRACTICE:** Write LC Q91 (Consecutive Numbers) using Gap & Island from scratch. | Write LC Q101 (Department Top 3 Salaries) using DENSE_RANK from scratch.

FAANG & Big Tech

Strategy + Questions

Company-by-company breakdown — exactly what each company tests

■ Google — Strategy & Questions

■ *Real interview experience: 'Google asked me to write a query to compute 7-day retention rate. They didn't give me table schemas — I had to ask clarifying questions first. Then they wanted me to optimize it. The key was identifying it as a self-join on month+1 pattern.'*

■ GOOGLE TESTS

Window functions (ROW_NUMBER, DENSE_RANK, LAG, LEAD), retention/engagement metrics, complex multi-table JOINS, recursive CTEs for hierarchies, gap & island for streaks. They always ask: 'How would you optimize this query?' and 'What if the table has 1 billion rows?'

#109 Nth Highest Salary

Google Interview

◆ MEDIUM

Google

■ **Problem:** Get the Nth highest distinct salary. Return NULL if fewer than N exist.

■ **Tables:** Employee(Id,Salary)

■ **Solution:**

```
WITH r AS (SELECT Salary, DENSE_RANK() OVER (ORDER BY Salary DESC) rnk FROM Employee)
SELECT MAX(Salary) AS NthHighestSalary FROM r WHERE rnk=3; -- replace 3 with N
```

■ INTERVIEW TRICK

Know 3 approaches: DENSE_RANK, MAX subquery, LIMIT OFFSET. Google expects you to know all 3.

■ INTERVIEW TRICK

INTERVIEW SCRIPT: 'I'll use DENSE_RANK because it handles ties correctly. If two people have salary 5000, they both count as rank 1, so rank 2 is the next distinct value.'

#110 User Retention Rate

Google Interview

▲ HARD

Google

■ **Problem:** % of users active in month M who were ALSO active in month M+1 (classic retention metric).

■ **Tables:** user_activity(user_id, activity_date)

■ **Solution:**

```
WITH monthly AS (SELECT DISTINCT user_id,DATE_FORMAT(activity_date,'%Y-%m') month FROM user_acti
vity),
retained AS (
  SELECT m1.month, COUNT(DISTINCT m1.user_id) retained
  FROM monthly m1 JOIN monthly m2 ON m1.user_id=m2.user_id
  AND m2.month=DATE_FORMAT(DATE_ADD(STR_TO_DATE(CONCAT(m1.month, '-01'), '%Y-%m-%d'),INTERVAL 1
MONTH), '%Y-%m')
  GROUP BY m1.month
)
SELECT r.month,ROUND(100.0*r.retained/COUNT(DISTINCT m.user_id),2) retention_rate
FROM retained r JOIN monthly m USING(month) GROUP BY r.month,r.retained;
```

■ INTERVIEW TRICK

D1 retention = next day. D7 = INTERVAL 6 DAY. D30 = INTERVAL 29 DAY. Same pattern, different interval.

■■ WATCH OUT

INTERVIEW: Always ask 'How do we define active?' before writing. Google specifically looks for this.

#111 Median Salary Per Department

Google Interview

▲ HARD

Google

■ **Problem:** Find the median salary for each department.

■ **Tables:** Employee(department_id, salary)

■ **Solution:**

```
WITH r AS (SELECT department_id,salary,
  ROW_NUMBER() OVER (PARTITION BY department_id ORDER BY salary) rn,
  COUNT(*) OVER (PARTITION BY department_id) cnt
  FROM Employee)
SELECT department_id, AVG(salary) median_salary
FROM r WHERE rn BETWEEN cnt/2.0 AND cnt/2.0+1 GROUP BY department_id;
```

■ KEY PATTERN

BETWEEN cnt/2.0 AND cnt/2.0+1: catches one middle row (odd) or two middle rows (even). Always /2.0!

■ Amazon — Strategy & Questions

■ *Real interview experience: 'Amazon asked me to find duplicate orders in their orders table. Simple question, but they watched how I handled edge cases — empty table, all duplicates, single row. They also asked how I would monitor this in production.'*

■ AMAZON TESTS

Running totals, revenue analysis, duplicate detection, date-range queries, window functions for ranking. Amazon loves practical data problems tied to e-commerce: orders, products, sellers, customers. They ask: 'How would you detect fraud?' 'How would you find top-selling products?'

#112 Running Total Revenue

Amazon Interview

● EASY

Amazon

■ **Problem:** For each order, compute cumulative revenue ordered by date.

■ **Tables:** orders(order_id,order_date,amount)

■ Solution:

```
SELECT order_id,order_date,amount,  
SUM(amount) OVER (ORDER BY order_date,order_id ROWS UNBOUNDED PRECEDING) running_total  
FROM orders ORDER BY order_date,order_id;
```

■ KEY PATTERN

ROWS UNBOUNDED PRECEDING = cumulative from start. Always use ROWS (not RANGE) for running totals.

■ INTERVIEW TRICK

INTERVIEW: 'I'm using ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW which gives a cumulative sum. I specify ROWS not RANGE to avoid issues when multiple orders have the same date.'

#113 Find Duplicate Orders

Amazon Interview

● EASY

Amazon

■ **Problem:** Find customers who placed multiple orders for same product on same day.

■ **Tables:** Orders(order_id,customer_id,product_id,order_date)

■ Solution:

```
SELECT customer_id,product_id,order_date,COUNT(*) AS duplicate_count  
FROM Orders GROUP BY customer_id,product_id,order_date HAVING COUNT(*)>1;
```

■ KEY PATTERN

Deduplication check: GROUP BY all identifying columns, HAVING COUNT > 1.

#114 7-Day Rolling Revenue

Amazon Interview

◆ MEDIUM

Amazon,Stripe

■ **Problem:** For each day, calculate the 7-day rolling average revenue.

■ **Tables:** sales(sale_date,revenue)

■ Solution:

```
SELECT sale_date,revenue,  
ROUND(AVG(revenue) OVER (ORDER BY sale_date ROWS BETWEEN 6 PRECEDING AND CURRENT ROW),2) rolling  
_7day_avg  
FROM sales ORDER BY sale_date;
```

■ KEY PATTERN

ROWS BETWEEN 6 PRECEDING AND CURRENT ROW = 7-row window (current + 6 before).

■ Meta/Facebook — Strategy & Questions

■ *Real interview experience: 'Meta asked me to compute DAU/MAU ratio. Simple on the surface, but they pushed: what if a user logs in multiple times a day? What's your DISTINCT strategy? Then: how do you handle the ratio if MAU is 0?'*

■ META TESTS

Product metrics (DAU, MAU, retention), user engagement queries, top-N per user (posts, comments, friends), histogram distributions, A/B test analysis. Scenarios always involve Facebook/Instagram product data. They specifically test: NULL handling, edge cases, DISTINCT usage.

#115 Monthly Active Users (MAU)

Meta Interview

◆ MEDIUM

Meta

■ **Problem:** Count MAUs in July 2022. Active = acted in BOTH June AND July.

■ **Tables:** user_actions(user_id,event_type,event_date)

■ **Solution:**

```
SELECT 7 month,COUNT(DISTINCT user_id) monthly_active_users FROM user_actions
WHERE MONTH(event_date)=7 AND YEAR(event_date)=2022
      AND user_id IN (SELECT DISTINCT user_id FROM user_actions
                      WHERE MONTH(event_date)=6 AND YEAR(event_date)=2022);
```

■ INTERVIEW TRICK

INTERVIEW: 'Before writing, I want to clarify: how do we define active? Any action, or specific actions?' Meta loves this question.

#116 Top 3 Posts Per User

Meta Interview

◆ MEDIUM

Meta

■ **Problem:** 3 most recent posts per user. Tie-break by post_id DESC.

■ **Tables:** posts(user_id,post_id,created_at)

■ **Solution:**

```
WITH r AS (SELECT user_id,post_id,created_at,
                 ROW_NUMBER() OVER (PARTITION BY user_id ORDER BY created_at DESC,post_id DESC) rn
            FROM posts)
SELECT user_id,post_id,created_at FROM r WHERE rn<=3;
```

■ WATCH OUT

Always add tie-breaker (post_id DESC) for determinism. Meta specifically tests if you remember this.

#117 Histogram of Comments Per User

Meta Interview

◆ MEDIUM

Meta

■ **Problem:** For each comment count value, how many users have that count? (January 2020)

■ **Tables:** users,fb_comments

■ **Solution:**

```
WITH uc AS (
  SELECT u.user_id,COUNT(fb.user_id) comment_count FROM users u
  LEFT JOIN fb_comments fb ON u.user_id=fb.user_id AND MONTH(fb.created_at)=1 AND YEAR(fb.create
d_at)=2020
  GROUP BY u.user_id
)
SELECT comment_count,COUNT(*) user_count FROM uc GROUP BY comment_count ORDER BY comment_count;
```

■ WATCH OUT

Date in ON clause (not WHERE) — ensures users with 0 January comments appear as count=0.

■ KEY PATTERN

HISTOGRAM = count per entity first, then group by those counts. Two-level aggregation.

■ Microsoft — Strategy & Questions

■ MICROSOFT TESTS

Hierarchical queries (recursive CTEs), employee org charts, window functions for business analytics, complex aggregations, date manipulation for reporting. Common in BI Engineer and Data Analyst roles.

#118 Employee Manager Hierarchy

Microsoft Interview

▲ HARD

Microsoft

■ **Problem:** For each employee, find the name of their top-level manager.

■ **Tables:** Employee(id,name,manager_id)

■ **Solution:**

```
WITH RECURSIVE h AS (  
  SELECT id,name,manager_id,name top_manager FROM Employee WHERE manager_id IS NULL  
  UNION ALL  
  SELECT e.id,e.name,e.manager_id,h.top_manager FROM Employee e JOIN h ON e.manager_id=h.id  
)  
SELECT name,top_manager FROM h ORDER BY name;
```

■ KEY PATTERN

Recursive CTE: ANCHOR=roots (no manager). RECURSIVE=join children to parents, carry top_manager.

■ MEMORY TRICK

MySQL: WITH RECURSIVE. PostgreSQL: WITH RECURSIVE. SQL Server: WITH (no RECURSIVE keyword).

■ Uber/Airbnb/Netflix/Stripe — Strategy & Questions

■ UBER/AIRBNB/NETFLIX/STRIPE TESTS

Sessionization, streak analysis, cohort retention, fraud detection, A/B test metrics, funnel analysis. These companies test SQL in context of their actual product data. Know Gap & Island, LAG/LEAD, running totals cold.

#119 Sessionization

Uber/Airbnb Interview

▲ HARD

Uber,Airbnb

■ **Problem:** Assign session_id to each event. New session if gap > 30 min from previous event per user.

■ **Tables:** page_events(user_id,event_time)

■ **Solution:**

```

WITH f AS (
  SELECT user_id,event_time,
    CASE WHEN TIMESTAMPDIFF(MINUTE,LAG(event_time) OVER (PARTITION BY user_id ORDER BY event_time),event_time)>30
      OR LAG(event_time) OVER (PARTITION BY user_id ORDER BY event_time) IS NULL
    THEN 1 ELSE 0 END new_session FROM page_events
),
s AS (SELECT user_id,event_time,SUM(new_session) OVER (PARTITION BY user_id ORDER BY event_time ROWS UNBOUNDED PRECEDING) session_id FROM f)
SELECT * FROM s;

```

■ KEY PATTERN

LAG detects gap>30min → flag 1. Cumulative SUM of flags = session_id (increments each new session).

■■ WATCH OUT

IS NULL check for first event per user (LAG returns NULL for first row) — always a new session.

#120 Longest Login Streak

Uber/Lyft Interview

▲ HARD

Uber, Lyft

■ **Problem:** Longest streak of consecutive daily logins per user.

■ **Tables:** logins(user_id,login_date)

■ **Solution:**

```

WITH d AS (SELECT DISTINCT user_id,DATE(login_date) login_date FROM logins),
g AS (SELECT user_id,login_date,login_date-INTERVAL ROW_NUMBER() OVER (PARTITION BY user_id ORDER BY login_date) DAY grp FROM d),
s AS (SELECT user_id,grp,COUNT(*) streak FROM g GROUP BY user_id,grp)
SELECT user_id,MAX(streak) longest_streak FROM s GROUP BY user_id;

```

■ KEY PATTERN

Deduplicate to daily level first. Gap & Island on dates: subtract ROW_NUMBER days from date.

==■ **PRACTICE:** For a Google interview: write a query to find users who logged in on all 7 days of last week. | For a Meta interview: write a query to compute the DAU/MAU ratio for each month.

SECTION 9

Service-Based Company Strategy

TCS · Infosys · Wipro · Accenture · Cognizant — Questions repeated every year

■ SERVICE-BASED COMPANY STRATEGY

These companies test FUNDAMENTALS, not advanced patterns. They care about: (1) Correct output. (2) Knowing the SQL syntax cleanly. (3) Knowing 2-3 approaches to each common problem. (4) Clear verbal explanation. If you master Sections 1–6, service company interviews will feel very easy. Focus: 2nd highest salary, self-joins, GROUP BY/HAVING, basic JOINS, NULL handling.

■ *Real experience: 'TCS asked me the same 5 questions that appear in every TCS SQL round: 2nd highest salary, employees earning more than manager, customers who never ordered, delete duplicates, and swap gender. I knew all 5 cold. Got selected. This PDF has all 5 and more.'*

TCS — Top 5 Most Repeated Questions

Infosys — Top Questions

Wipro & Accenture — Key Questions

#121 Second Highest Salary

TCS (Most Asked)

● EASY

TCS,Infosys,Wipro,
ALL

■ **Problem:** Find second highest salary. Return NULL if not exists.

■ **Tables:** Employee(Id,Salary)

■ **Solution:**

```
-- Approach 1 (most common expected answer)
SELECT MAX(Salary) SecondHighestSalary FROM Employee WHERE Salary<(SELECT MAX(Salary) FROM Employee);
-- Approach 2: DENSE_RANK (handles ties)
WITH r AS (SELECT Salary,DENSE_RANK() OVER (ORDER BY Salary DESC) rk FROM Employee)
SELECT MAX(CASE WHEN rk=2 THEN Salary END) SecondHighestSalary FROM r;
-- Approach 3: LIMIT OFFSET
SELECT DISTINCT Salary FROM Employee ORDER BY Salary DESC LIMIT 1 OFFSET 1;
```

■■ WATCH OUT

Know ALL 3 approaches! TCS/Infosys interviewers ALWAYS ask 'do you know another way?'

■ PRO TIP

Approach 1 and 2 return NULL when no 2nd highest exists. Approach 3 returns empty set.

#122 Employees Earning More Than Manager

TCS,ALL

● EASY

TCS,Infosys,Wipro,
ALL

■ **Problem:** Find employees who earn more than their direct manager.

■ **Tables:** Employee(id,name,salary,managerId)

■ **Solution:**

```
SELECT e.name AS Employee FROM Employee e
JOIN Employee m ON e.managerId=m.id
WHERE e.salary>m.salary;
```

■ **MEMORY TRICK**

Classic self-join: e=employee, m=manager. INNER JOIN correctly excludes top-level employees (NULL managerId).

■ **INTERVIEW TRICK**

INTERVIEW SCRIPT: 'I'm doing a self-join on the Employee table — aliasing it twice. e is the employee, m is their manager. I join on e.managerId = m.id, then filter where employee salary > manager salary.'

#123 Customers Who Never Ordered

TCS,Infosys

● EASY

TCS,Infosys,Cognizant

■ **Problem:** Find all customers who have never placed any order.

■ **Tables:** Customers(id,name), Orders(id,customerId)

■ **Common Mistake:**

```
-- WRONG: NOT IN trap with NULLs
WHERE id NOT IN (SELECT customerId FROM Orders)
-- If any customerId is NULL → returns 0 rows!
```

■ **Solution:**

```
SELECT c.name FROM Customers c
LEFT JOIN Orders o ON c.id=o.customerId
WHERE o.id IS NULL;
```

■ **KEY PATTERN**

Anti-join. LEFT JOIN + IS NULL = rows in left with NO match in right.

#124 Swap Gender Values

TCS,Accenture

● EASY

TCS,Accenture

■ **Problem:** Swap all 'm' to 'f' and 'f' to 'm' in a SINGLE query.

■ **Tables:** Salary(id,name,sex,salary)

■ **Common Mistake:**

```
-- WRONG: Two-step approach overwrites all values!
UPDATE Salary SET sex='f' WHERE sex='m'; -- m→f
UPDATE Salary SET sex='m' WHERE sex='f'; -- ALL f→m, including original m's!
```

■ **Solution:**

```
UPDATE Salary SET sex=CASE WHEN sex='m' THEN 'f' ELSE 'm' END;
```

■ **KEY PATTERN**

CASE inside UPDATE. Single atomic operation. No temp variable needed.

#125 Find Nth Highest Salary

TCS,Wipro

◆ MEDIUM

TCS,Wipro,Accenture

■ **Problem:** Find the Nth highest salary (N=3 example).

■ **Solution:**

```
SELECT DISTINCT Salary FROM Employee ORDER BY Salary DESC LIMIT 1 OFFSET 2; -- OFFSET=N-1
-- DENSE_RANK (handles ties, recommended)
WITH r AS (SELECT Salary,DENSE_RANK() OVER (ORDER BY Salary DESC) rk FROM Employee)
SELECT MAX(Salary) FROM r WHERE rk=3;
```

■ PRO TIP

LIMIT 1 OFFSET N-1 is simple but mishandles ties. DENSE_RANK is always more correct.

#126 Find Duplicate Emails

Infosys

● EASY

Infosys,Cognizant

■ **Problem:** Find employees who share their email with at least one other.

■ **Tables:** Employee(id,name,email)

■ **Solution:**

```
SELECT name,email FROM Employee
WHERE email IN (SELECT email FROM Employee GROUP BY email HAVING COUNT(*)>1)
ORDER BY email;
```

■ KEY PATTERN

Find duplicate email values (HAVING>1), then show all employees with those emails.

#127 Department Max Employees

Infosys,Wipro

◆ MEDIUM

Infosys,Wipro

■ **Problem:** Find the department(s) with the most employees. Handle ties.

■ **Tables:** Employee,Department

■ **Solution:**

```
WITH dc AS (SELECT department_id,COUNT(*) emp_count FROM Employee GROUP BY department_id)
SELECT d.name,dc.emp_count FROM dc JOIN Department d ON dc.department_id=d.id
WHERE dc.emp_count=(SELECT MAX(emp_count) FROM dc);
```

■ PRO TIP

WHERE emp_count = MAX(...) handles ties. LIMIT 1 would miss tied departments.

#128 Consecutive Absent Students

Wipro,Accenture

◆ MEDIUM

Wipro,Accenture

■ **Problem:** Find students absent for 3+ consecutive days.

■ **Tables:** Attendance(student_id,absent_date,is_absent)

■ **Solution:**

```
WITH g AS (  
  SELECT student_id,absent_date,  
         absent_date-INTERVAL ROW_NUMBER() OVER (PARTITION BY student_id ORDER BY absent_date) DAY grp  
  FROM Attendance WHERE is_absent=1  
)  
SELECT student_id,MIN(absent_date) from_date,MAX(absent_date) to_date,COUNT(*) days  
FROM g GROUP BY student_id,grp HAVING COUNT(*)>=3 ORDER BY student_id,from_date;
```

■ **KEY PATTERN**

Gap & Island on dates. Same pattern as SQL Project Planning (Q51) and Consecutive Numbers (Q91).

#129 Total Sales Per Region Month

TCS,Cognizant

● EASY

TCS,Cognizant

■ **Problem:** Total sales grouped by region and month.

■ **Tables:** Sales(region,sale_date,amount)

■ **Solution:**

```
SELECT region,DATE_FORMAT(sale_date,'%Y-%m') month,SUM(amount) total_sales  
FROM Sales GROUP BY region,month ORDER BY region,month;
```

■ **MEMORY TRICK**

DATE_FORMAT('%Y-%m')='YYYY-MM'. Format codes: %Y=year, %m=month, %d=day, %H=hour.

#130 Students Not Enrolled Any Course

Infosys,TCS

● EASY

Infosys,TCS

■ **Problem:** Find all students not enrolled in any course.

■ **Tables:** Students,Enrollments

■ **Solution:**

```
SELECT s.student_id,s.name FROM Students s  
LEFT JOIN Enrollments e ON s.student_id=e.student_id  
WHERE e.student_id IS NULL;
```

■ **KEY PATTERN**

Anti-join. Always use LEFT JOIN + IS NULL for 'find X with no Y'. Cleaner and safer than NOT IN.

—■ **PRACTICE:** Write Q122 (Employees > Manager) from scratch — this is asked at TCS, Infosys, Wipro, Accenture, every year. | Write Q124 (Swap Gender) in a single UPDATE — most candidates get this wrong.

Top 50 Most Frequently Asked SQL Questions

Asked across ALL companies worldwide — theory + coding — with complete answers

■ PRO TIP

These 50 questions are based on frequency analysis of thousands of real SQL interviews at companies from startups to Fortune 500. If you can answer all 50 confidently, you are in the top 5% of SQL interview candidates in the world.

#	Question	Type	Companies	Answer
1	What is SQL and why is it important?	Theory	ALL	SQL (Structured Query Language) is the standard language for managing relational databases. It is used to create, read, update, and delete data. It is the #1 skill for data analysts, data engineers, and backend developers. Used at every company that stores data.
2	What is the difference between SQL and NoSQL?	Theory	ALL	SQL databases are relational (tables with rows/columns, fixed schema, ACID compliance): MySQL, PostgreSQL, Oracle. NoSQL databases are non-relational (flexible schema, horizontal scaling): MongoDB, Cassandra, Redis. SQL = structured, consistent. NoSQL = flexible, scalable for unstructured data.
3	Explain DELETE, TRUNCATE, DROP differences.	Theory	ALL	DELETE: row-by-row, DML, WHERE supported, rollback possible. TRUNCATE: all rows instantly, DDL, no WHERE, no rollback (MySQL), resets auto-increment. DROP: removes entire table structure + data. Most destructive.
4	What are ACID properties?	Theory	ALL	Atomicity (all or nothing), Consistency (valid state always), Isolation (transactions don't interfere), Durability (committed data survives crashes).
5	What is the difference between WHERE and HAVING?	Theory	ALL	WHERE filters rows BEFORE grouping (no aggregates). HAVING filters groups AFTER GROUP BY (aggregates allowed).
6	What is a PRIMARY KEY vs UNIQUE KEY vs FOREIGN KEY?	Theory	ALL	PRIMARY KEY: unique + NOT NULL + only one per table + clustered index. UNIQUE KEY: unique + allows one NULL + multiple per table. FOREIGN KEY: references PK of another table, enforces referential integrity.
7	What are the different types of JOINS?	Theory	ALL	INNER (matching rows only), LEFT (all left + matches), RIGHT (all right + matches), FULL OUTER (all rows both sides), CROSS (all combinations), SELF (table joined to itself).
8	What is a VIEW? What is a MATERIALIZED VIEW?	Theory	ALL	VIEW: virtual table, no data stored, always fresh, query runs on access. MATERIALIZED VIEW: physical snapshot on disk, fast reads, must be refreshed, may be stale.
9	What is Normalization? Explain 1NF, 2NF, 3NF.	Theory	ALL	1NF: atomic values. 2NF: no partial dependency. 3NF: no transitive dependency. Mnemonic: 'The key, the whole key, and nothing but the key.'

10	What is UNION vs UNION ALL?	Theory	ALL	UNION: removes duplicates (slower, requires sort). UNION ALL: keeps all rows (faster). Use UNION ALL unless you explicitly need deduplication.
11	What is a CTE (WITH clause)?	Theory	FAANG	Named temporary result set. Improves readability, replaces complex subqueries. Recursive CTEs traverse hierarchical data.
12	What are Window Functions?	Theory	FAANG	Calculations across related rows WITHOUT collapsing them. ROW_NUMBER, RANK, DENSE_RANK, LAG, LEAD, SUM OVER, AVG OVER.
13	What is a correlated vs non-correlated subquery?	Theory	ALL	Non-correlated: runs once independently. Correlated: references outer query, runs once per outer row (slower). Replace correlated with JOINS or CTEs when possible.
14	What is an index? How does it work?	Theory	ALL	Data structure (B-Tree) that speeds up data retrieval. Trade-off: faster reads, slower writes. Left-prefix rule for composite indexes.
15	What is the SQL execution order?	Theory	FAANG	FROM/JOIN → WHERE → GROUP BY → HAVING → SELECT → DISTINCT → ORDER BY → LIMIT
16	What is RANK vs DENSE_RANK vs ROW_NUMBER?	Theory	FAANG	ROW_NUMBER: always unique (1,2,3). RANK: gaps after ties (1,1,3). DENSE_RANK: no gaps (1,1,2).
17	What is the difference between stored procedure and function?	Theory	Service-based	Procedure: CALL, can do DML, multiple result sets, not usable in SELECT. Function: returns one value, used in SELECT/WHERE, deterministic.
18	What is NULL in SQL?	Theory	ALL	Unknown/missing value. NULL = NULL is NULL (not TRUE). Any math with NULL = NULL. Use IS NULL, COALESCE, IFNULL to handle.
19	What is CHAR vs VARCHAR?	Theory	Service-based	CHAR(n): fixed-length, faster for fixed data. VARCHAR(n): variable-length, flexible.
20	What is EXPLAIN in SQL?	Theory	FAANG	Shows the query execution plan — which indexes are used, join types, estimated rows scanned. Run EXPLAIN on every complex query to find performance bottlenecks.
21	Write a query to find the 2nd highest salary.	Coding	ALL	SELECT MAX(Salary) FROM Employee WHERE Salary < (SELECT MAX(Salary) FROM Employee);
22	Write a query to find duplicate emails.	Coding	ALL	SELECT email FROM Employee GROUP BY email HAVING COUNT(*) > 1;
23	Write a query to delete duplicate records keeping the smallest id.	Coding	ALL	DELETE p1 FROM Person p1 JOIN Person p2 ON p1.email=p2.email AND p1.id>p2.id;
24	Write a query to find employees earning more than their manager.	Coding	ALL	SELECT e.name FROM Employee e JOIN Employee m ON e.managerId=m.id WHERE e.salary>m.salary;
25	Find customers who never placed an order.	Coding	ALL	SELECT c.name FROM Customers c LEFT JOIN Orders o ON c.id=o.customerId WHERE o.id IS NULL;
26	Find the top 3 salaries in each department.	Coding	FAANG	WITH r AS (SELECT name,dept,salary,DENSE_RANK() OVER (PARTITION BY dept ORDER BY salary DESC) rk FROM Employee) SELECT * FROM r WHERE rk<=3;
27	Write a running total query.	Coding	FAANG	SELECT date,amount,SUM(amount) OVER (ORDER BY date ROWS UNBOUNDED PRECEDING) AS running_total FROM sales;
28	Write a 7-day moving average query.	Coding	FAANG	SELECT date,AVG(revenue) OVER (ORDER BY date ROWS BETWEEN 6 PRECEDING AND CURRENT ROW) AS moving_avg FROM sales;

29	Find consecutive numbers appearing 3+ times.	Coding	FAANG	WITH g AS (SELECT num,id-ROW_NUMBER() OVER (PARTITION BY num ORDER BY id) grp FROM Logs) SELECT DISTINCT num FROM g GROUP BY num,grp HAVING COUNT(*)>=3;
30	Find the median salary.	Coding	FAANG	WITH r AS (SELECT salary,ROW_NUMBER() OVER (ORDER BY salary) rn,COUNT(*) OVER() cnt FROM Employee) SELECT AVG(salary) FROM r WHERE rn BETWEEN cnt/2.0 AND cnt/2.0+1;
31	Swap all male/female values in one query.	Coding	Service-based	UPDATE Salary SET sex=CASE WHEN sex='m' THEN 'f' ELSE 'm' END;
32	Find customers who bought product A but not B.	Coding	ALL	SELECT DISTINCT customer_id FROM orders WHERE product_id=1 AND customer_id NOT IN (SELECT customer_id FROM orders WHERE product_id=2);
33	Write a query to pivot data (crosstab).	Coding	FAANG	SELECT MAX(CASE WHEN Occupation='Doctor' THEN Name END) Doctor, MAX(CASE WHEN Occupation='Singer' THEN Name END) Singer FROM (SELECT Name,Occupation,ROW_NUMBER() OVER (PARTITION BY Occupation ORDER BY Name) rn FROM OCCUPATIONS) t GROUP BY rn;
34	Find the longest consecutive login streak.	Coding	FAANG	WITH g AS (SELECT user_id,login_date,login_date-INTERVAL ROW_NUMBER() OVER (PARTITION BY user_id ORDER BY login_date) DAY grp FROM logins), s AS (SELECT user_id,grp,COUNT(*) streak FROM g GROUP BY user_id,grp) SELECT user_id,MAX(streak) FROM s GROUP BY user_id;
35	Write a sessionization query (new session if gap > 30 min).	Coding	FAANG	WITH f AS (SELECT *,CASE WHEN TIMESTAMPTDIFF(MINUTE,LAG(event_time) OVER (PARTITION BY user_id ORDER BY event_time),event_time)>30 OR LAG(event_time) OVER (PARTITION BY user_id ORDER BY event_time) IS NULL THEN 1 ELSE 0 END ns FROM events) SELECT *,SUM(ns) OVER (PARTITION BY user_id ORDER BY event_time ROWS UNBOUNDED PRECEDING) session_id FROM f;
36	Find employees with salary above department average.	Coding	ALL	SELECT e.name,e.salary,e.department FROM Employee e WHERE e.salary>(SELECT AVG(salary) FROM Employee WHERE department=e.department);
37	Find the department with the highest average salary.	Coding	ALL	SELECT department,AVG(salary) avg_sal FROM Employee GROUP BY department ORDER BY avg_sal DESC LIMIT 1;
38	Write a recursive CTE to traverse an org chart.	Coding	FAANG	WITH RECURSIVE h AS (SELECT id,name,manager_id,name top_mgr FROM Employee WHERE manager_id IS NULL UNION ALL SELECT e.id,e.name,e.manager_id,h.top_mgr FROM Employee e JOIN h ON e.manager_id=h.id) SELECT name,top_mgr FROM h;
39	Find users who logged in every day for the last 7 days.	Coding	FAANG	SELECT user_id FROM logins WHERE login_date >= CURDATE()-INTERVAL 6 DAY GROUP BY user_id HAVING COUNT(DISTINCT DATE(login_date))=7;
40	Calculate year-over-year revenue growth.	Coding	FAANG	SELECT year,revenue,LAG(revenue) OVER (ORDER BY year) prev_year_revenue,ROUND(100.0*(revenue-LAG(revenue) OVER (ORDER BY year))/LAG(revenue) OVER (ORDER BY year),2) yoy_growth FROM revenue_by_year;
41	Find the first purchase date per customer.	Coding	ALL	SELECT customer_id,MIN(order_date) first_purchase FROM Orders GROUP BY customer_id;

42	Find customers who purchased in both month 1 and month 2.	Coding	ALL	SELECT customer_id FROM Orders WHERE MONTH(order_date)=1 INTERSECT SELECT customer_id FROM Orders WHERE MONTH(order_date)=2; -- Or use self-join for MySQL
43	Write a query to get the cumulative percentage.	Coding	FAANG	SELECT name,score,ROUND(100.0*SUM(score) OVER (ORDER BY score DESC ROWS UNBOUNDED PRECEDING)/SUM(score) OVER(),2) cumulative_pct FROM scores;
44	Find the weighted average rating per product.	Coding	ALL	SELECT product_id,SUM(rating*reviews)/SUM(reviews) AS weighted_avg_rating FROM product_ratings GROUP BY product_id;
45	Find all managers (employees who manage someone).	Coding	Service-based	SELECT DISTINCT manager_id AS manager FROM Employee WHERE manager_id IS NOT NULL;
46	Find employees hired in the last 30 days.	Coding	Service-based	SELECT * FROM Employee WHERE hire_date >= CURDATE()-INTERVAL 30 DAY;
47	Find the most common value in a column (mode).	Coding	ALL	SELECT salary FROM Employee GROUP BY salary ORDER BY COUNT(*) DESC LIMIT 1;
48	Find records where a column value starts with a number.	Coding	Service-based	SELECT * FROM products WHERE product_code REGEXP '^[0-9]';
49	Write a query using EXISTS.	Coding	ALL	SELECT name FROM Customers c WHERE EXISTS (SELECT 1 FROM Orders o WHERE o.customer_id=c.id AND o.amount>1000);
50	Find the top seller per region.	Coding	FAANG	WITH r AS (SELECT region,seller_id,SUM(amount) total,ROW_NUMBER() OVER (PARTITION BY region ORDER BY SUM(amount) DESC) rn FROM sales GROUP BY region,seller_id) SELECT region,seller_id,total FROM r WHERE rn=1;

The Master

Cheat Sheet

Memorize this — it is your secret weapon in every interview

1. SQL Execution Order (Visual)

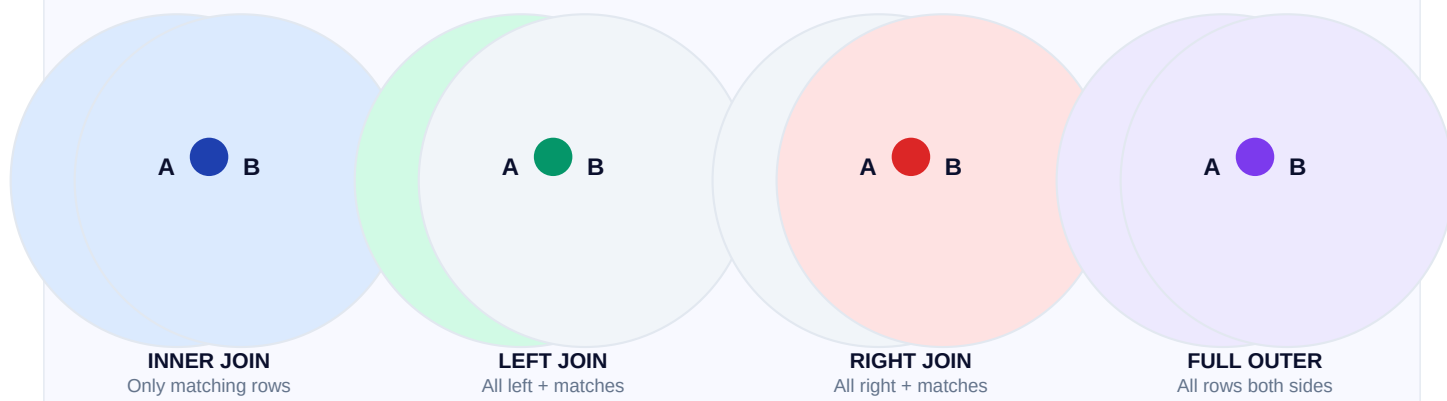
SQL Execution Order — SQL Does NOT Execute Top to Bottom!



■ You WRITE: SELECT...FROM...WHERE — SQL READS: FROM first, WHERE second, SELECT fifth!

2. JOIN Types (Visual)

SQL JOIN Types — Visual Guide



3. NULL Truth Table (Visual)

NULL — SQL Uses 3-Valued Logic: TRUE / FALSE / NULL

NULL = NULL	→ NULL (not TRUE!) Use IS NULL	5 + NULL	→ NULL. Any math with NULL = NULL
NOT IN with NULL	→ EMPTY RESULT! Use NOT EXISTS	TRUE AND NULL	→ NULL (unknown)
FALSE AND NULL	→ FALSE	TRUE OR NULL	→ TRUE
COUNT(*)	→ counts NULLs. COUNT(col) skips NULLs	COALESCE(a, b)	→ returns first non-NULL value

4. Window Functions Reference

```
-- SYNTAX
function() OVER (
  PARTITION BY group_col  -- optional: reset per group (like GROUP BY but keeps rows)
  ORDER BY  sort_col      -- optional: order within partition
  ROWS BETWEEN frame_start AND frame_end
)
-- KEY FRAMES
ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW  -- running/cumulative total
ROWS BETWEEN 6 PRECEDING AND CURRENT ROW           -- 7-row rolling window
ROWS BETWEEN 1 PRECEDING AND 1 FOLLOWING            -- 3-row centered window
```

Function	Output	Use Case	Ties?
ROW_NUMBER()	1,2,3,4,5	Dedup, top-1 per group, pagination	Always unique
RANK()	1,1,3,4	Sports ranking (gap after ties)	Gaps after ties
DENSE_RANK()	1,1,2,3	Top-N salaries — NO gaps after ties	No gaps
NTILE(n)	1,1,2,2,3,3	Quartiles, percentile buckets	n buckets
LAG(col,n,default)	prev row value	MoM change, day-over-day comparison	—
LEAD(col,n,default)	next row value	Forecast, future comparison	—
SUM(col) OVER	cumulative sum	Running total, partition total	—
AVG(col) OVER	rolling average	7-day avg, moving average	—
FIRST_VALUE(col)	first in window	First event per session/group	—
LAST_VALUE(col)	last in window	Need UNBOUNDED FOLLOWING frame!	—

5. The 15 Universal SQL Interview Patterns

■ MEMORY TRICK

Every SQL interview question in the world maps to one of these 15 patterns. When you see a new question: IDENTIFY THE PATTERN FIRST. Then write the SQL. This is the skill that separates top candidates from average candidates.

#	Pattern Name	SQL Approach	Real Example
1	Top-N per Group	ROW_NUMBER/DENSE_RANK OVER (PARTITION BY group ORDER BY val DESC), WHERE rn<=N	Top 3 salaries per dept
2	Running/Cumulative Total	SUM() OVER (ORDER BY date ROWS UNBOUNDED PRECEDING)	Cumulative revenue
3	Moving/Rolling Window	AVG() OVER (ORDER BY date ROWS BETWEEN N PRECEDING AND CURRENT ROW)	7-day rolling avg
4	Gap & Island (Consecutive Runs)	col - ROW_NUMBER() OVER (PARTITION BY val ORDER BY col) AS grp	Consecutive logins, absent days

5	Anti-Join (Find No-Match)	LEFT JOIN + WHERE right.id IS NULL OR NOT EXISTS	Customers who never ordered
6	Median	ROW_NUMBER+COUNT, WHERE rn BETWEEN total/2.0 AND total/2.0+1	Median salary
7	Histogram / Frequency	Step1: COUNT per entity. Step2: GROUP BY that count	Users by comment count
8	Pivot / Crosstab	MAX(CASE WHEN cat='X' THEN val END) GROUP BY row_id	Monthly sales by category
9	Recursive Hierarchy	WITH RECURSIVE: anchor=roots, recursive=join children to parents	Org chart, category tree
10	Period Comparison (YoY)	LAG(metric) OVER (ORDER BY period) OR self-join on period+1	MoM growth
11	Sessionization	LAG gap>threshold → flag → CUMSUM of flags = session_id	Web sessions
12	Relational Division ('ALL Y')	GROUP BY X, HAVING COUNT(DISTINCT Y) = (SELECT COUNT(*) FROM Y)	Bought all products
13	Weighted Average	SUM(value×weight)/SUM(weight) NOT AVG(value)	Avg selling price
14	Deduplication / Duplicates	GROUP BY key cols, HAVING COUNT(*)>1 finds duplicates	Duplicate emails
15	First/Last per Group	ROW_NUMBER per group ORDER BY date, filter rn=1 OR MIN/MAX	First purchase, latest price

6. Index Optimization Quick Reference

Rule	Detail
■ Composite index order	Equality column first, range column second. (country, salary) for WHERE country='IN' AND salary>50000
■ Covering index	All queried columns in index → no heap lookup. CREATE INDEX idx ON t(a,b,c) for SELECT a,b WHERE c=x
■ Function on indexed column	WHERE YEAR(date)=2024 kills index. Rewrite: WHERE date BETWEEN '2024-01-01' AND '2024-12-31'
■ Leading wildcard	LIKE '%John' cannot use index. LIKE 'John%' CAN (prefix search)
■ Low cardinality column	Index on gender(M/F) is ignored by optimizer. Index high-selectivity columns.
■ Left-prefix rule	Index (A,B,C): A, A+B, A+B+C queries benefit. B alone does NOT use this index.
■ Always EXPLAIN	Run EXPLAIN SELECT... Check 'type': ref>range>index>ALL. type=ALL = full scan = problem.
■ Write trade-off	Every index = faster reads but slower writes (INSERT/UPDATE/DELETE must maintain all indexes).

7. Interview Day Checklist — Do These in EVERY SQL Interview

- When shown a problem: identify the PATTERN first (15 patterns in this book). Say it out loud.

- Ask clarifying questions: 'Can this column be NULL?' / 'How do we define active user?' / 'What should happen with ties?'
- Say your thinking: 'I'm using ROW_NUMBER partitioned by department because this is a Top-N per group problem...'
- Mention edge cases: 'What if the table is empty? My query returns NULL which is correct.' / 'What about ties?'
- Offer optimization: 'For large tables, an index on X would make this query much faster'
- Know 2 approaches: 'I can also solve this with a CTE instead of a subquery — would you like to see that?'
- Write clean SQL: Proper indentation, meaningful aliases (e=employee, m=manager), explicit ASC/DESC
- Test with example: trace through 2-3 rows mentally to verify your query is correct before submitting

You now hold everything you need to become the best SQL candidate in any interview room in the world. 8 visual diagrams that make concepts permanently clear. All 58 HackerRank questions. All 50 LeetCode SQL questions. Real FAANG questions with company-specific strategies. Service-based company questions that are asked every year. The 50 most frequently asked SQL questions across all companies. The 15 universal patterns that solve ANY SQL question. There is only ONE gap left between this PDF and your dream job: Close this book. Open a blank editor. Write every solution yourself. Do that, and no SQL interview in the world can stop you. ■